ORIUS

Observation–Reality Integrity for Universal Safety

A Universal Safety Monograph for Physical AI

Runtime Safety Under Degraded Observation

Pratik Niroula

Minnesota State University, Mankato

CIT Major, Mathematics Minor

A universal runtime safety layer developed across battery, autonomous vehicle, industrial, healthcare, navigation, and aerospace domains.

ORIUS Safety Principle

"Physical AI safety is bounded by observation reliability."

April 2026

Abstract

Physical AI systems do not act on reality directly. They act on measurements, filtered state estimates, delayed telemetry, and learned summaries of the world. When that observation surface degrades, an action can remain admissible on the observed state while already being unsafe on the true state. This dissertation treats that discrepancy as an action-semantics safety problem rather than as a prediction problem alone, and names it the observation–action safety gap (OASG).

Existing lines of work address important parts of this failure mode: conformal uncertainty quantifies predictive spread, safe control constrains actions, runtime assurance supervises release, and fault monitoring detects degraded telemetry. What is still missing is the integrated runtime object that connects those pieces at the moment of physical release. ORIUS, Observation–Reality Integrity for Universal Safety, is proposed as that missing object: a certificate-aware runtime safety layer for physical AI under degraded observation.

The dissertation develops ORIUS through three implementation-grounded surfaces. First, `DomainAdapter` makes the domain safety predicate, fallback semantics, and repair geometry explicit. Second, a universal benchmark schema turns degraded-observation replay into a common evaluation contract across domains. Third, CertOS provides certificate lifecycle, audit continuity, and release semantics so that repaired action can be governed as a runtime object rather than reported only as controller-side logic. The formal contribution is a domain-agnostic theorem ladder connecting degraded observation, reliability-conditioned uncertainty, repaired action, fallback, and certificate semantics under explicit assumptions.

Empirically, the dissertation establishes one deep witness row and several bounded multi-domain instantiations without changing what ORIUS means at the release boundary. Battery provides the strongest theorem-to-runtime closure, and industrial process control plus healthcare monitoring support defended bounded rows. The result is a thesis that is universal in architecture, tiered in evidence: the autonomous-vehicle row is a defended bounded runtime row under the current artifact surface, while canonical raw-data closure remains incomplete, and equal-domain closure remains open because navigation and aerospace are still gated. The manuscript therefore separates what is proved, what is validated, and what remains

roadmap-only. That separation is enforced not only by prose but by a claim-locked publication surface in which parity matrices, runtime and governance tables, and manifest-backed figures govern what the dissertation is allowed to say.

Acknowledgments

This thesis was assembled as a solo research project around one question: how should a physical-AI system behave when the observation channel is no longer trustworthy, but the consequences of action remain physical and irreversible? ORIUS was built to answer that question with a runtime contract rather than a domain-specific controller.

The deepest current evidence still comes from the battery witness row because it offers the cleanest theorem-to-code-to-artifact surface in the repository. The broader contribution of the book, however, depends on carrying that safety grammar across autonomous vehicles, industrial control, healthcare monitoring, navigation, and aerospace without letting the prose outrun the evidence.

The open-source systems, datasets, and research communities behind the methods cited in this book made that synthesis possible. Their work is visible throughout the bibliography, the artifact index, and the reviewer-facing appendices that helped harden the final manuscript.

Contents

Abstract i

Acknowledgments iii

I Problem, Principle, and Gap 1

1 Introduction and Thesis Claims 2

1.1 The hidden law of physical AI . . . 2
1.2 Why degraded observation is a release-boundary problem . . . 3
1.3 Thesis claim . . . 3
1.4 Why this dissertation is a monograph, not a stitched paper bundle . . . 4
1.5 Main contributions . . . 5
1.6 Claim discipline . . . 6
1.7 Why ORIUS matters beyond one witness domain . . . 6
1.8 Dissertation organization . . . 7

2 Related Work and the Novelty Gap 8

2.1 Conformal prediction and adaptive calibration . . . 8
2.2 Runtime assurance and Simplex-style supervision . . . 9
2.3 Safety filters, shields, and robust safe control . . . 9
2.4 Anomaly, drift, and degraded-observation detection . . . 9
2.5 Domain-specific cyber-physical safety systems . . . 10
2.6 Safety-case governance and lifecycle discipline . . . 10
2.7 Benchmark, replay, and reproducibility gaps . . . 10
2.8 Why ORIUS is not another forecasting or perception paper . . . 11
2.9 The novelty gap ORIUS addresses . . . 11

3 Formal Problem Formulation **13**
3.1 State, observation, action, and constraints . . . 13
3.2 Observation degradation operator and fault process . . . 13
3.3 Fault taxonomy . . . 14
3.4 Definition of the observation–action safety gap . . . 14
3.5 Observation-consistent state sets . . . 15
3.6 Repair, fallback, and release . . . 15
3.7 Dual-trajectory evaluation . . . 16
3.8 DomainAdapter grounding . . . 16
3.9 Scope and assumptions . . . 17

II ORIUS Framework and Theorem Ladder **18**

4 ORIUS Architecture: DC3S Runtime Layer **19**
4.1 Why a runtime layer is the right object . . . 19
4.2 The DC3S release step . . . 20
4.3 Universal runtime flow . . . 20
4.4 DomainAdapter as the universal boundary . . . 20
4.5 Relationship to supervision and safety filters . . . 22
4.6 Certificate emission and operational semantics . . . 22
4.7 Runtime cost and operational realism . . . 23
4.8 Architectural takeaway . . . 23

5 Mathematical Foundations and Safety Guarantees **24**
5.1 Why the theorem surface is intentionally narrow . . . 24
5.2 Notation and assumption discipline . . . 24
5.3 Definition of OASG as a formal event . . . 25
5.4 The theorem ladder . . . 25
5.5 Risk envelope interpretation . . . 26
5.6 Why the proofs stay battery-backed but the objects stay universal . . . 27
5.7 Theorem-to-artifact linkage . . . 27
5.8 What is proved, validated, and still open . . . 27

6 Benchmark and Reproducibility Protocol **28**
6.1 Why ORIUS-Bench is central to the thesis . . . 28
6.2 Dual-trajectory evaluation . . . 28
6.3 Deterministic replay and fault schedules . . . 29

6.4 Metrics that match the runtime object 29
6.5 Strict universal mode and legacy compatibility 29
6.6 Artifact provenance and claim locking 30
6.7 Reproducibility checklist 30
6.8 Benchmark takeaway 31

III Witness Domain Evidence 32

7 Witness Domain Implementation 33
7.1 Why battery is the witness domain 33
7.2 Witness-domain problem and plant semantics 33
7.3 Data surfaces and dataset cards 34
7.4 Forecasting, calibration, and reliability surfaces 34
7.5 Control baselines and adapter implementation 34
7.6 Locked artifact protocol 36
7.7 Witness-row scope 36

8 Witness Results and Failure Analysis 37
8.1 The governing empirical question 37
8.2 Experiment design and baseline logic 37
8.3 Headline result 38
8.4 Mechanistic interpretation 38
8.5 Ablations and where the gain comes from 38
8.6 Conditional coverage and reliability slices 39
8.7 Operational traces and certificate-aware behavior 40
8.8 Bounded deep-learning novelty inside the witness row 40
8.9 What fails without the runtime layer 41
8.10 Witness-row claim boundary 41

IV Universalization and Extensions 42

9 Universal ORIUS Across Domains 43
9.1 What universality means in this dissertation 43
9.2 A transfer recipe rather than a portability slogan 43
9.3 Cross-domain status at a glance 44
9.4 Domain synthesis 44

9.5 Architecture universality versus parity universality . . . 45
9.6 Default proof mode for the dissertation . . . 46
9.7 Boundary of the universality claim . . . 46

10 Temporal Certificates and Graceful Degradation 48
10.1 Certificate horizon as a runtime object . . . 48
10.2 Why temporal safety belongs inside runtime assurance . . . 48
10.3 Expiration, half-life, and safe-hold . . . 49
10.4 Severe-blackout protocol . . . 49
10.5 Graceful degradation policies . . . 49
10.6 Temporal extension . . . 50

11 Compositional Safety 51
11.1 Why local safety does not automatically compose . . . 51
11.2 A simple counterexample . . . 51
11.3 Shared constraints as the right composition surface . . . 51
11.4 Multi-agent certificates and negotiation . . . 52
11.5 Why composition remains bounded in the dissertation . . . 52
11.6 Compositional takeaway . . . 52

12 CertOS Runtime Assurance 53
12.1 Why CertOS belongs in the main argument . . . 53
12.2 Lifecycle operations . . . 53
12.3 Runtime invariants . . . 54
12.4 Audit chain and post hoc verification . . . 54
12.5 Governance alignment, not compliance . . . 55
12.6 Universality and evidence-tiered maturity . . . 55
12.7 Governance boundary . . . 55

V Submission Boundary 56

13 Governance, Reproducibility, and Conclusion 57
13.1 What is proved . . . 57
13.2 What is empirically validated . . . 57
13.3 What is out of scope . . . 58
13.4 Reproducibility discipline . . . 58
13.5 Threats to validity . . . 58

13.6 Conclusion 59

a Assumption Register **60**

b Proofs **62**

b.1 Proof map 62

b.2 Proof sketches 63

b.3 Interpretation 64

c Benchmark Specifications **65**

c.1 Canonical fault families 65

c.2 Combined profile: `drift_combo` 67

c.3 Replay and fault-injection implementation 67

c.4 Severity mapping 67

d Artifact and Claim Index **69**

d.1 Claim classes and governing surfaces 69

d.2 Headline claim families 70

d.3 Canonical manuscript-facing evidence 70

d.4 Interpretation 71

e Safety-Case Bundle Index **72**

e.1 How to read the bundle index 73

e.2 Interpretive boundary 73

Extended Reading and Index **74**

List of Tables

4.1 Domain state schema for ORIUS universal framework. 22
4.2 Safety predicates per domain. 22
4.3 Universal DC3S pipeline stages. 23

5.1 DC3S Theorem Dependency and Proof Completeness. Each formal result is implemented in code and tested by at least one automated test. 26

6.1 Canonical universal replay output. Battery is the locked reference row; industrial and healthcare are proof-validated; AV is bounded proof-validated under the TTC contract; navigation is shadow-synthetic; aerospace is experimental. 30

7.1 Dataset-card inventory for the conference release family. 34
7.2 Promoted six-model forecasting comparison. Deep learning baselines evaluate point-forecast performance, confirming GBM's operational dominance in this tabular regime. Conformal uncertainty artifacts (PICP, Width) are materialized exclusively for the production-candidate model (GBM).. *— = n/a; PICP@90 and Width only for GBM.* 35
7.3 Battery DeepOQE diagnostics on the held-out degraded-telemetry episodes. 36

8.1 Main safety and cost outcomes under telemetry faults. 38
8.2 Ablation study with strict gate fields and statistical tests. 39
8.3 Regime-group CQR coverage and interval width summary. 39

9.1 Equal-domain parity gate for the universal-first ORIUS book. 44

11.1 Multi-Agent Fleet Coordination: joint safety violations and useful work under shared transformer capacity constraint (limit = 80 MW, two batteries at 60 MW each = 120 MW proposed). Independent protocol violates; ORIUS-coordinated eliminates violations. 52

a.1 ORIUS dissertation assumption register. 60

b.1 Theorem-to-proof map for the ORIUS dissertation. 62

c.1 Witness-domain fault families used to instantiate the universal ORIUS-Bench telemetry-degradation contract. 66
c.2 `drift_combo` default parameter values. 67
c.3 Fault-to-pipeline severity mapping. 68

d.1 Claim classes and their governing artifact surfaces. 69
d.2 Headline claim families and their canonical evidence surfaces. 70

e.1 Safety-case bundle index for the ORIUS dissertation. 72

List of Figures

1.1 ORIUS as a universal runtime safety layer between nominal intelligence and physical release. The five-stage kernel recurs across domains even though the domains do not share a plant model, nominal controller family, or optimization objective. 5

4.1 ORIUS runtime flow from degraded telemetry to repaired action and governed certificate. The figure is generated from the same publication assets that feed the manuscript-facing parity and governance surfaces. 21

5.1 The integrated theorem gate keeps theorem language aligned to typed contracts, assumptions, and locked artifacts rather than to informal prose alone. 27

7.1 Witness-domain reliability baselines. The engineered forecasting surface and the learned reliability surface are intentionally reported together so the dissertation does not confuse forecasting novelty with degraded-observation novelty. 35

8.1 Battery learned-reliability safety surface. The deep model is judged by true-state safety, intervention burden, and certificate-valid release behavior rather than by forecast error alone. 40

9.1 Cross-domain ORIUS validation surface. One runtime grammar carries across six domains, while each row remains limited by its own defended evidence surface. 46

10.1 Blackout half-life surface used to motivate certificate expiration, bounded safe-hold, and temporal intervention discipline. 49

10.2 Four-policy graceful degradation comparison. The dissertation uses this family to treat fallback quality as a benchmarked layer instead of an unexamined emergency default. 50

12.1 CertOS lifecycle and runtime-governance surface across ORIUS domains. The figure shows how certificate issue, validation, expiry, fallback, and audit continuity are exposed as part of runtime safety rather than as post hoc reporting. 54

Part I

Problem, Principle, and Gap

Chapter 1

Introduction and Thesis Claims

A battery dispatch stack can continue releasing power while feeder telemetry has already gone stale enough to hide a true state-of-charge or network violation. A vehicle controller can preserve lane-centering behavior even after the scene estimate has fallen behind the physical traffic geometry. A bedside monitoring policy can remain internally consistent while its input stream smooths away the very transition that should have triggered intervention. These systems look different at the level of plant dynamics and domain semantics, but they share one hidden law: *physical AI acts on observations, not on reality.* Once the observation surface degrades, a system can continue to compute coherent actions while the physical basis for those actions has already failed.

This dissertation is organized around that release-boundary hazard. Its central claim is that physical-AI safety under degraded observation requires a dedicated runtime layer between nominal intelligence and actuation, and that this layer can be expressed through one reusable grammar: *Detect*, *Calibrate*, *Constrain*, *Shield*, and *Certify*. ORIUS is the implemented form of that grammar. It is not proposed as a universal controller, not as a replacement for domain-specific estimation or control, and not as a blanket claim that every supported domain has already reached equal empirical maturity. It is proposed as the right runtime object for deciding what action remains admissible once observation trust degrades.

1.1 The hidden law of physical AI

Physical AI systems never act on truth directly. They act on packets, sensor fusion products, state estimators, learned summaries, and communication layers that compress the world into a runtime observation object. That compression is unavoidable. It is what allows downstream forecasting, planning, optimization, and control to happen at all. But it also creates a standing safety debt: the legality of an action is usually evaluated on the observed state even

though the true physical state may already have diverged.

The dissertation therefore takes a position that differs from the standard performance-first framing. The deepest safety problem is not only forecast error, perception error, or nominal control error. It is the possibility that a system remains computationally coherent while its observation surface is no longer trustworthy enough to justify physical release. In ORIUS, that discrepancy is named the *observation–action safety gap* (OASG). The OASG is the primary scientific object because it marks the exact point at which internally legal behavior can become physically unsafe without obvious software failure.

1.2 Why degraded observation is a release-boundary problem

Most deployed stacks already include pieces of the response: anomaly scores, confidence thresholds, conservative margins, or fallback modes. What they often lack is a single runtime discipline that ties those pieces together. Detection alone is not enough, because a system that detects degraded telemetry must still decide how uncertainty changes. Wider uncertainty is not enough, because the admissible action set must be recomputed under that widened state set. Recomputed admissibility is not enough, because unsafe candidate actions must be repaired or replaced before release. And none of that is enough if the runtime cannot later explain why an action was allowed, for how long, and under which fallback or certificate state.

ORIUS is organized around that full loop. The contribution of the dissertation is therefore not "better forecasting" and not "better control" in isolation. It is a runtime safety layer that accepts the reality of degraded observation and then turns that reality into governed release semantics. This is why the central ORIUS question is not "what prediction model is best?" but "what action remains safe to release, with what uncertainty, with what repair, and under what certificate, once observation quality degrades?"

1.3 Thesis claim

The thesis claim can be stated compactly:

> *Physical AI under degraded observation requires a certificate-aware runtime safety layer that mediates between nominal intelligence and physical release; that layer can be expressed through one reusable contract of reliability assessment, uncertainty*

inflation, admissible-set tightening, repaired or fallback action, and governed certificate emission.

ORIUS operationalizes that claim through a typed Detect–Calibrate–Constrain–Shield–Certify kernel.

- **Detect** scores the reliability of the observation channel.
- **Calibrate** widens the observation-consistent state set as reliability falls.
- **Constrain** recomputes admissible action under that widened uncertainty surface.
- **Shield** repairs unsafe candidate action or escalates to a fallback policy.
- **Certify** records why release remains admissible, for how long, and under what governance state.

The key point is architectural. ORIUS is written as a post-nominal layer. It wraps inherited domain controllers rather than assuming they can all be redesigned from first principles. That choice is what makes the framework portable across batteries, vehicles, industrial control, healthcare monitoring, navigation, and aerospace without pretending that those domains share one plant model or one optimal nominal controller.

1.4 Why this dissertation is a monograph, not a stitched paper bundle

The argument of the dissertation is structurally single-threaded. It begins with one hidden hazard, develops one runtime object, formalizes one theorem ladder, measures one benchmark contract, and then asks how far that same safety grammar travels across multiple domains. Because the argument is unified, the correct dissertation form is a doctrine-driven monograph rather than a stitched collection of papers. Battery remains the deepest witness row, but it is not the identity of the thesis. Its role is to provide the deepest theorem-to-code-to-artifact surface against which the broader universal claim can be calibrated.

Three implementation-grounded objects make that monograph possible. First, the `DomainAdapter` contract fixes what every domain must expose: telemetry semantics, constraint predicates, repair hooks, and certificate payloads. Second, ORIUS-Bench fixes the dual-trajectory replay contract through which observed safety and true-state safety can be compared. Third, CertOS turns repaired action into a governed release object through lifecycle transitions, certificate horizons, and audit records. Taken together, those objects let the dissertation defend a reusable runtime layer rather than a loose set of case studies.

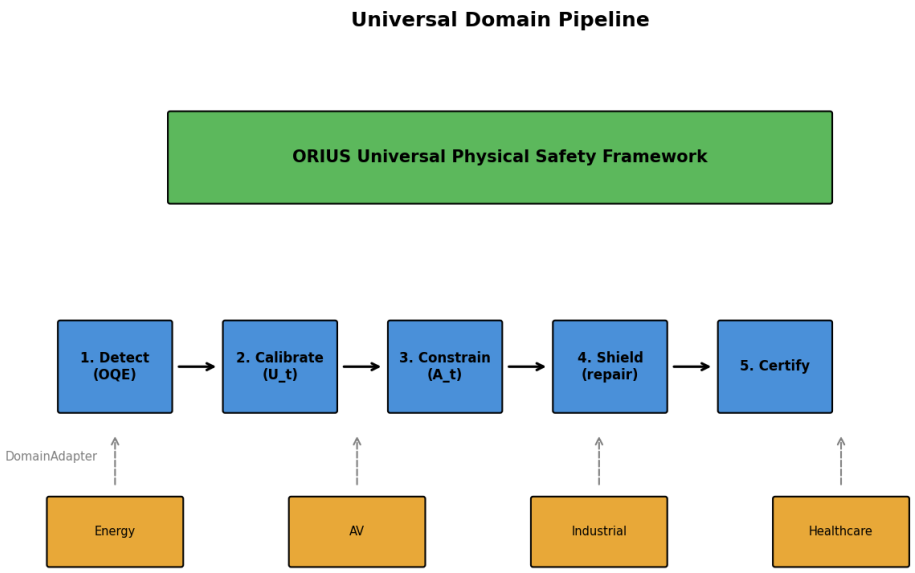

Figure 1.1: ORIUS as a universal runtime safety layer between nominal intelligence and physical release. The five-stage kernel recurs across domains even though the domains do not share a plant model, nominal controller family, or optimization objective.

1.5 Main contributions

The dissertation makes four contributions.

1. **A problem contribution.** It defines degraded observation as a universal release-boundary hazard for physical AI through the observation–action safety gap.

2. **A systems contribution.** It presents ORIUS as a typed runtime layer that connects reliability assessment, uncertainty inflation, safe-set tightening, repair or fallback, and governed certificate emission.

3. **A formal contribution.** It develops a theorem bridge from reliability-aware uncertainty and repaired action to bounded reduction in true-state safety violation under explicit assumptions.

4. **An empirical contribution.** It assembles a claim-locked benchmark and multi-domain evidence program that distinguishes a deepest witness row from defended bounded peers, portability rows, and explicitly gated rows rather than flattening them into a false equal-maturity claim.

1.6 Claim discipline

The dissertation uses three claim classes and keeps them visibly separate from the first chapter forward.

Proved claims are theorem-grade statements under explicit assumptions about degraded observation, uncertainty inflation, repair, and bounded violation reduction.

Validated claims are empirical statements tied to replayable artifacts, runtime traces, and governed benchmark outputs.

Roadmap claims are structurally motivated extensions that are intentionally not promoted as defended evidence.

This distinction is not only stylistic caution. It prevents a universal architectural argument from being mistaken for equal empirical parity, and it keeps the manuscript aligned with the repository's claim-lock discipline. Reviewers should be able to tell, at any point in the book, whether a statement is theorem-backed, artifact-backed, or explicitly left as future work.

1.7 Why ORIUS matters beyond one witness domain

The importance of ORIUS is not that batteries, vehicles, industrial plants, healthcare systems, navigation stacks, and aerospace controllers secretly reduce to one plant. The importance is that they all face the same release-boundary question once observation trust degrades: what action can still be justified, with what uncertainty, with what fallback, and under what certificate? ORIUS freezes the semantics of that question so that cross-domain comparison happens at the level of runtime safety rather than only at the level of application area.

That framing also changes how the thesis relates to adjacent literature. Conformal prediction and adaptive calibration provide disciplined uncertainty surfaces [1–3]. Runtime assurance contributes supervisory release logic [4, 5]. Safety filters and barrier methods provide computable constraint-preserving action geometry [6–8]. Safety-case governance and risk management standards clarify why claims, lifecycle discipline, and auditable evidence matter even when formal compliance is not being claimed [9–12]. ORIUS brings those strands together by centering the missing object they all touch but do not fully stabilize: the runtime safety layer for degraded observation.

1.8 Dissertation organization

The rest of the dissertation follows the doctrine directly. Chapters 2 and 3 define the novelty gap and the formal OASG object. Chapters 4 through 6 present the ORIUS architecture, theorem ladder, and benchmark contract. Chapters 7 and 8 defend the battery witness row. Chapters 9 through 12 ask how far the same runtime grammar carries across domains, temporal certificate semantics, composition, and governance. Chapter 13 then states the precise submission boundary, reproducibility discipline, and final thesis claim.

Boundary of this chapter. This chapter claims only the dissertation's governing doctrine and contribution structure. It does not yet prove the theorem ladder or validate equal-domain parity; those later claims remain tied to the benchmark, artifact, and governance surfaces that follow.

Chapter 2

Related Work and the Novelty Gap

ORIUS enters a crowded intellectual landscape, but the relevant literatures are crowded in a revealing way. Each contributes a necessary piece of the final argument: uncertainty quantification explains what can be said about predictive trust, runtime assurance explains why intervention must remain separate from nominal control, safe-control methods explain how constraints can reshape action, fault-detection work explains how degradation is recognized, and governance frameworks explain how claims and evidence must be bounded. The novelty gap appears only when those families are read together. They are rich in local technique yet still leave the release-boundary object of degraded observation split across separate traditions.

2.1 Conformal prediction and adaptive calibration

Conformal prediction and related distribution-free uncertainty methods provide one of the few disciplined ways to wrap learned models with coverage-aware uncertainty without assuming a perfectly specified error law [1–3, 13–15]. Conformalized quantile regression and adaptive conformal variants extend that logic to heteroskedastic and nonstationary settings, which makes them especially relevant once a physical system moves through regime changes or degraded telemetry states [2, 3, 16–20].

What remains missing is the release-boundary question that follows from calibrated uncertainty. Coverage guarantees do not by themselves decide how the admissible action set should change, which fallback is legitimate, or what evidence must accompany release once the observation surface becomes unreliable. ORIUS therefore treats calibration as an input to runtime safety rather than as the full safety object.

2.2 Runtime assurance and Simplex-style supervision

Runtime assurance architectures, supervisory controllers, shielding, and runtime verification supply the language of intervention, veto, fallback, and monitored execution [4, 5, 21–27]. That line of work is indispensable because it treats safe release as a separate problem from nominal policy design.

Yet most supervisory schemes enter after the nominal controller has already proposed an action under a state estimate that is treated as sufficiently meaningful. ORIUS addresses an earlier fault line: once observation quality degrades, the assurance layer must reason jointly about reliability, widened state uncertainty, repaired action, fallback, and certificate semantics. A supervisor without that observation-side logic is still missing the central OASG object.

2.3 Safety filters, shields, and robust safe control

Control barrier functions, predictive safety filters, robust MPC, and tube-based control translate model uncertainty into admissible control sets and corrective intervention rules [6–8, 28–35]. These methods are the closest classical relatives of ORIUS because they formalize the difference between nominal control intent and safe control execution.

The remaining gap is that this family usually assumes the uncertainty set and safety object have already been defined in a domain-specific way. ORIUS does not replace that work with a universal controller. Instead, it provides a typed runtime contract that can host many repair mechanisms while keeping reliability estimation, fallback semantics, and certificate discipline explicit. The novelty is not the existence of safe control alone; it is the runtime composition that makes safe control portable across domains.

2.4 Anomaly, drift, and degraded-observation detection

Fault diagnosis, change detection, cyber-physical security, and anomaly monitoring provide the vocabulary for stale, delayed, reordered, dropped, or corrupted observations [36–46]. This literature is essential because degraded observation must first be recognized before it can influence action release.

Detection alone does not specify how action semantics should change after trust degrades. A drift alarm can say that something is wrong without saying how uncertainty inflates, how the safe set tightens, what repair is permitted, or what certificate makes the next step auditable. ORIUS uses detection as the opening move in a runtime safety sequence, not as

the full response.

2.5 Domain-specific cyber-physical safety systems

Energy systems, autonomous mobility stacks, industrial control, healthcare monitoring, and other cyber-physical systems already contain rich safety practice in their own local forms [5, 6, 36, 38–40]. This practice matters because any claim of universality that ignores domain structure is empty from the start.

What they usually do not expose is a common contract through which degraded-observation replay, repaired action, fallback semantics, certificate validity, and latency can be compared across domains. ORIUS addresses that missing cross-domain runtime object rather than trying to erase domain structure.

2.6 Safety-case governance and lifecycle discipline

The dissertation also inherits an important lesson from safety-case and lifecycle practice: strong safety claims require structured argument, explicit assumptions, traceable evidence, and disciplined lifecycle boundaries. ORIUS does not claim compliance with sector-specific standards, but its claim-lock pipeline, artifact provenance, and certificate-governance surfaces are best understood in conversation with claim-based safety arguments and lifecycle-oriented risk management [9–12].

Governance frameworks say how evidence should be organized and how claims should be bounded; they do not themselves provide the runtime object that converts degraded observation into repaired or denied release. ORIUS borrows the discipline of explicit evidence structure and lifecycle boundaries, but its technical novelty remains the runtime layer that sits between observation degradation and actuation.

2.7 Benchmark, replay, and reproducibility gaps

There is also a measurement gap that cuts across the technical literatures above. Many papers report prediction error, control regret, collision rate, or intervention count, but do not record the dual-trajectory evidence needed to tell whether an action was only observed-safe or was truly safe on the latent physical state. That omission matters because the OASG is invisible unless the benchmark keeps both trajectories in view at once.

Without replayable fault schedules, governed intervention traces, and a locked mapping from headline claims to artifacts, a physical-AI safety result is difficult to audit and easy to

overstate. ORIUS therefore treats measurement discipline as part of the contribution rather than as packaging around the contribution. The benchmark contract, claim matrix, and manifest surfaces are necessary because they keep the monograph grounded in true-state safety rather than nominal software coherence.

2.8 Why ORIUS is not another forecasting or perception paper

The literature also invites a narrower but common misunderstanding: that ORIUS should be read as one more attempt to beat a baseline predictor, one more perception stack, or one more domain-specific robust controller. The dissertation rejects that framing directly. In the strongest witness row, forecasting quality matters because it affects calibrated uncertainty and therefore the release boundary, but the main scientific object is still the runtime layer that decides what action remains admissible once the observation surface becomes unreliable.

Why this matters for the novelty claim. This distinction prevents ORIUS from collapsing into a model-comparison paper. A new forecaster or perception backbone may improve nominal error while leaving the release-boundary question unchanged. ORIUS is novel only if it remains centered on degraded observation, admissible repair, governed fallback, and certificate semantics. That is why the related-work chapter is organized by missing runtime objects rather than by model families alone.

2.9 The novelty gap ORIUS addresses

The literature becomes most powerful when these families are read together, but it also reveals a persistent separation of levels. Uncertainty methods stop at calibrated sets. Assurance methods stop at supervisory intervention. Safe-control methods stop at plant-specific repair geometry. Detection methods stop at identifying degraded telemetry. Domain systems stop at application-local safety practice. What is missing between them is a single typed runtime composition that starts with observation reliability and ends with repaired action, fallback, certificate semantics, and cross-domain replay under one common contract.

That missing composition is ORIUS's novelty claim. The dissertation does not argue that any one ORIUS component is unprecedented in isolation. It argues that the combination of observation reliability, action repair, certificate semantics, and universal replay has not previously been stabilized as one reusable runtime safety layer for physical AI. ORIUS is

therefore not "better forecasting," "better perception," or "one more controller." It is a release-boundary discipline for degraded-observation safety.

The closest plausible rival to ORIUS is not any single paper in these families. It is an imagined composition of them: adaptive conformal intervals for uncertainty, a safety filter or Simplex supervisor for intervention, anomaly detection for degraded telemetry, and runtime verification for logging and policy enforcement. Even that composite stack still stops short of ORIUS's central object. It lacks a typed release contract that binds observed-state and true-state semantics, repaired action, fallback, certificate lifecycle, and cross-domain replay under one explicit interface. That final integration step is the novelty claim the dissertation defends.

Boundary of this chapter. This chapter establishes a literature gap, not a theorem or a parity result. Its role is to show why uncertainty, supervision, safe control, detection, and governance still leave the release-boundary object fragmented. The next chapters must still prove and validate that ORIUS closes enough of that fragmentation to justify a universal runtime safety-layer claim under bounded evidence.

Chapter 3

Formal Problem Formulation: OASG under Degraded Observation

3.1 State, observation, action, and constraints

Let $x_t^\star$ denote the true physical state of the plant at time t, let $\hat{x}_t$ denote the observation available to the nominal policy, and let u_t denote the candidate action proposed for release. Let $\mathcal{C}(x)$ denote the set of admissible actions when the state is x, or equivalently the safety predicate that separates legal from illegal actuation. ORIUS is concerned with regimes in which the software stack continues to evaluate legality on $\hat{x}_t$ even though the physically relevant state is $x_t^\star$ or a broader observation-consistent set around it.

This distinction matters because physical safety is defined on the plant, not on the software representation alone. If $\hat{x}_t$ lags, omits, or distorts the relevant state variables, then the nominal controller can continue producing actions that are legal with respect to the observed state while being unsafe with respect to the state that is actually realized. The formal problem of the dissertation is therefore not generic uncertainty. It is the mismatch between observation-conditioned legality and physical legality.

3.2 Observation degradation operator and fault process

Observation degradation is modeled as a runtime operator

$$\hat{x}_t = \mathcal{D}(x_t^\star; \delta_t),$$

where δ_t is a fault process that summarizes the degradation mechanisms affecting the telemetry channel. In the repository and benchmark, this process includes stale packets,

missing samples, delay, packet reordering, spikes, drift, transport corruption, and domain-specific state-estimation loss. The universal object of the dissertation is therefore not a plant alone; it is the coupled pair $(x_t^\star, \hat{x}_t)$ together with the degradation process that can make those two states diverge.

Two implications follow. First, the fault process is not merely a nuisance variable to be averaged away in nominal performance statistics. It is part of the safety problem definition. Second, runtime safety cannot be reduced to point estimation of the true state. Under degraded observation, the runtime must reason over a set of plausible physical states rather than rely on one best guess.

3.3 Fault taxonomy

The fault process is treated generically, but the benchmark and adapters instantiate it through a shared taxonomy:

1. **Freshness faults**: stale or frozen telemetry that preserves coherence while erasing temporal validity.
2. **Availability faults**: missing packets, dropped samples, and partial channel loss.
3. **Timing faults**: delay, jitter, or reordering that breaks synchronization with the physical plant.
4. **Value faults**: spikes, outliers, drift, bias, or corruption that distort the observed state itself.
5. **Inference faults**: estimator or learned-model errors that turn degraded inputs into overconfident downstream summaries.

This taxonomy is intentionally broad enough to travel across all ORIUS domains. A battery controller and an aerospace controller do not share the same raw sensors, but they do share the possibility that freshness, timing, value, or inference degradation breaks the semantics of observed legality.

3.4 Definition of the observation–action safety gap

The central event of the dissertation is the observation–action safety gap:

$$u_t \in \mathcal{C}(\hat{x}_t) \qquad \text{but} \qquad u_t \notin \mathcal{C}(x_t^\star).$$

Equivalently, the action appears safe on the observed state but unsafe on the true physical state. This event is what the benchmark later measures through true-state safety violation and observed-state satisfaction. It is also what makes degraded observation a first-class runtime problem rather than an upstream modeling nuisance.

The OASG formulation is stronger than a generic claim that "uncertainty matters." Uncertainty can be large without creating a safety violation if the action remains conservative enough. The OASG is the specific regime where the observation channel misleads the release decision itself. That is why ORIUS focuses on action mediation, not only on uncertainty reporting.

3.5 Observation-consistent state sets

ORIUS does not assume that the true state can be recovered exactly at runtime. Instead, it works with an observation-consistent state set

$$\mathcal{X}_t = \{x : x \text{ remains plausible given } \hat{x}_t \text{ and runtime reliability } w_t\},$$

where $w_t \in [0, 1]$ is the observation reliability score. As reliability degrades, $\mathcal{X}_t$ widens. This widening is the formal bridge between degraded telemetry and safe action. Once $\mathcal{X}_t$ expands, the admissible action set must contract:

$$\mathcal{U}_t^{\text{safe}} = \{u : u \in \mathcal{C}(x) \text{ for every } x \in \mathcal{X}_t\}.$$

The runtime question is then precise: given a candidate action u_t, does it belong to the current safe set $\mathcal{U}_t^{\text{safe}}$? If not, can it be repaired by projection or replacement? If not, which governed fallback policy must be triggered? This formulation is what lets ORIUS connect reliability scoring, uncertainty inflation, safe-set tightening, and certificate semantics into one runtime loop.

3.6 Repair, fallback, and release

Let Π_t denote the runtime repair map. Given a candidate action u_t, ORIUS computes either a repaired action

$$\tilde{u}_t = \Pi_t(u_t, \mathcal{X}_t)$$

that lies inside $\mathcal{U}_t^{\text{safe}}$, or else escalates to a fallback action u_t^{fb} specified by the domain governance policy. The formal point is that the runtime layer does not merely observe

illegality; it mediates release. This distinguishes ORIUS from alarm-only monitoring and from nominal predictive systems that surface confidence but leave the release decision unchanged.

The release object is completed by a certificate

$$\kappa_t = (\hat{x}_t, w_t, \mathcal{X}_t, \tilde{u}_t, h_t, g_t),$$

where h_t denotes a validity horizon or half-life proxy and g_t denotes governance metadata such as lifecycle state, fallback reason, or audit hash context. Later chapters make that certificate operational through CertOS.

3.7 Dual-trajectory evaluation

The dissertation evaluates ORIUS on dual trajectories. The *observed trajectory* is the one visible to the nominal controller and therefore to the candidate action it proposes. The *true trajectory* is the one used to determine whether that candidate or repaired action actually violates the physical safety envelope. A benchmark that inspects only observed-state legality cannot see the OASG directly. ORIUS-Bench therefore records the candidate action, the repaired action, true-state violation, observed-state satisfaction, optional true and observed margins, fallback and intervention markers, latency, and certificate validity inside one replay schema.

This dual-trajectory formulation is the critical measurement move of the dissertation. It keeps the benchmark aligned with the formal problem object instead of allowing nominal performance metrics to impersonate safety evidence.

3.8 DomainAdapter grounding

The universal formulation is grounded in the repository through the `DomainAdapter` contract. Each domain must supply:

1. telemetry parsing and state semantics,
2. uncertainty or margin construction hooks,
3. admissible-action or repair logic,
4. true-state violation predicates,
5. observed-state satisfaction predicates, and

6. certificate payload specifics.

This boundary is what makes the universal claim scientifically credible. The dissertation does not claim that one plant model or one controller solves every domain. It claims that every domain can expose the same runtime safety objects and thereby enter the same benchmark and governance discipline.

3.9 Scope and assumptions

The formal problem is intentionally narrower than a claim of general AI safety. ORIUS assumes:

- an actionable physical system with explicit or computable safety constraints,
- an observation channel that can degrade in ways that matter to release semantics,
- a runtime interposition layer capable of intercepting, repairing, or replacing actions,
- and a governance layer capable of recording why a release remained admissible.

ORIUS does not assume perfect state recovery, universal optimal control, or equal empirical maturity across all domains. It also does not assume that uncertainty calibration alone proves safety. The later theorem and validation chapters are therefore written under explicit assumptions and are tied to specific artifact surfaces rather than left as narrative implication.

Boundary of this chapter. This chapter defines the formal objects that the rest of the dissertation uses: degraded observation, observation-consistent state sets, repaired release, and dual-trajectory evaluation. It does not yet prove the theorem ladder or claim equal cross-domain closure; those claims remain bounded by the assumptions and evidence surfaces introduced in the chapters that follow.

Part II

ORIUS Framework and Theorem Ladder

Chapter 4

ORIUS Architecture: DC3S Runtime Layer

ORIUS enters the dissertation at the narrowest and most consequential point in the control loop: after nominal intelligence has proposed an action, but before that action is allowed to alter a physical plant. That placement is deliberate. Batteries, vehicles, industrial systems, healthcare monitors, navigation stacks, and aerospace controllers do not share one estimator, one optimizer, or one plant model. What they can share is a disciplined runtime treatment of what happens when degraded observation changes the meaning of safe release.

4.1 Why a runtime layer is the right object

The architectural claim is therefore narrower and stronger than a proposal for a universal controller. Degraded observation creates a release-boundary problem that sits between inference and actuation. Once telemetry becomes stale, delayed, missing, or corrupted, the relevant question is no longer only "what would the nominal controller do?" It becomes "what action can still be justified for physical release under the observation quality now available?" ORIUS is the layer that answers that second question.

That framing is what lets the book stay universal without drifting into abstraction. The runtime layer leaves domain-specific controllers, estimators, and plant models in place while freezing one common control-flow grammar for degraded observation. That grammar is Detect, Calibrate, Constrain, Shield, and Certify.

4.2 The DC3S release step

The universal runtime step has five stages.

Detect compute observation reliability from freshness, missingness, delay, ordering, spikes, and consistency markers;

Calibrate widen the observation-consistent state set as reliability falls;

Constrain recompute the admissible action set under the widened uncertainty set;

Shield repair the candidate action or replace it with a bounded fallback when no admissible repaired action remains;

Certify emit a causal certificate describing the reliability state, uncertainty summary, repaired action, fallback status, and lifecycle metadata that govern release.

What matters is not that each stage is unprecedented in isolation. The architectural move is the typed composition of all five at one runtime boundary. ORIUS treats degraded observation as something that must change release semantics immediately, not as a warning signal that can be logged while nominal actuation proceeds unchanged.

4.3 Universal runtime flow

Figure 4.1 makes the runtime object concrete. Reliability is not a dashboard number layered on top of an unchanged controller. It changes the admissible state set, which changes the admissible action set, which in turn changes whether nominal action may be released, whether it must be repaired, or whether the runtime must fall back to a safer policy. Certification closes the loop by recording what was released, why it remained admissible, and when that justification should expire.

4.4 DomainAdapter as the universal boundary

The `DomainAdapter` contract is the narrow interface that makes universality credible. The adapter owns the local semantics that should remain local:

- how raw telemetry is interpreted into state features;
- how true-state violation and observed-state satisfaction are defined;
- how candidate action is repaired or projected;

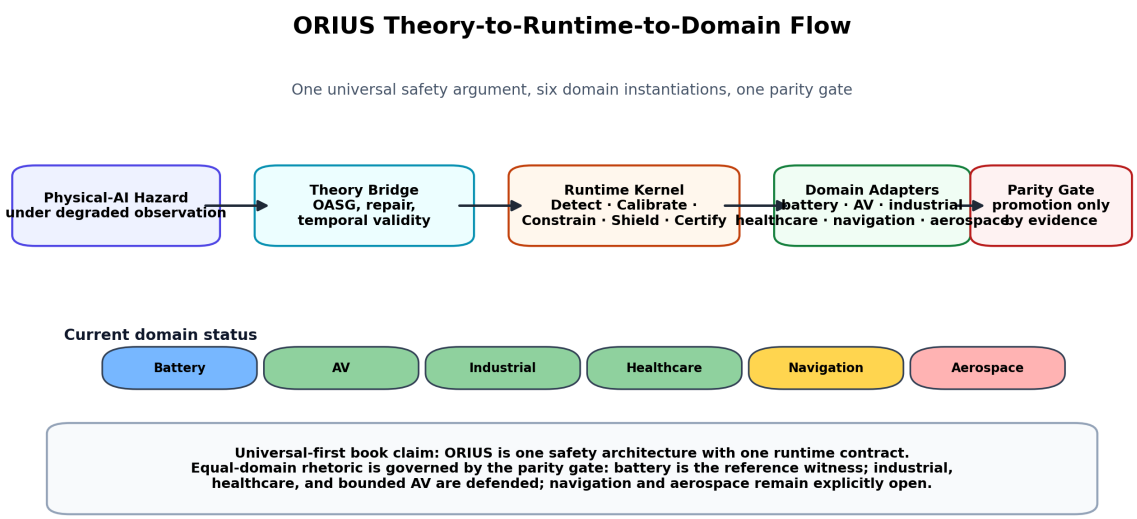

Figure 4.1: ORIUS runtime flow from degraded telemetry to repaired action and governed certificate. The figure is generated from the same publication assets that feed the manuscript-facing parity and governance surfaces.

- what fallback means for the plant; and
- what certificate payload must be preserved for reviewable release.

The universal kernel owns what should remain shared:

- reliability handling;
- uncertainty inflation;
- admissible-set tightening;
- benchmark-schema compatibility; and
- certificate lifecycle discipline.

This separation is the implementation answer to the universality question. ORIUS does not argue that every domain shares one plant geometry. It argues that each domain can expose its own geometry through one runtime contract while the safety logic surrounding degraded observation remains reusable.

Table 4.1: Domain state schema for ORIUS universal framework.

Domain	State Fields	Unit	Des
Energy	soc_mwh, load_mw, renewables_mw	MWh, MW	Bat
AV	position_m, speed_mps, speed_limit_mps, lead_position_m	m, m/s	Lon
Navigation	x, y, vx, vy	m, m/s	2D
Industrial	temp_c, pressure_mbar, humidity_pct, power_mw	°C, mbar, %, MW	Pro
Healthcare	hr_bpm, spo2_pct, respiratory_rate	bpm, %, /min	Vita
Aerospace	altitude_m, airspeed_kt, bank_angle_deg, fuel_remaining_pct	m, kt, deg, %	Flig

Table 4.2: Safety predicates per domain.

Domain	Safety Predicate	Config Key
Energy	SOC $\in$ [min_soc, max_soc]; charge/discharge mutual exclusion	min_soc_mwh
AV	$v \leq v_{\max}$; $x_{\text{lead}} - x \geq d_{\min}(v)$	speed_limit_mps
Navigation	(x, y) remains inside arena bounds and outside obstacle set	arena_min, arena_max
Industrial	temp $\in [\mathrm{T}_{\min}, \mathrm{T}_{\max}]$; power $\in [0, \mathrm{P}_{\max}]$	power_max_mw
Healthcare	hr $\in [\mathrm{hr}_{\min}, \mathrm{hr}_{\max}]$; $\mathrm{SpO}_2 \geq \mathrm{spo2}_{\min}$	spo2_min_pct
Aerospace	$v \in [\mathrm{v}_{\min}, \mathrm{v}_{\max}]$; $\lvert\mathrm{bank}\rvert \leq \mathrm{bank}_{\max}$; fuel $\geq \mathrm{fuel}_{\min}$	v_min_kt, max_bank_

4.5 Relationship to supervision and safety filters

ORIUS should be read as compatible with, not opposed to, established safety-filter and runtime-assurance ideas. Control barrier functions, predictive safety filters, and robust MPC show how plant constraints can be translated into admissible control updates [6–8, 31]. Simplex-style runtime assurance shows how a high-performance controller can be supervised by a safer fallback authority when operating conditions invalidate the nominal control contract [4, 5].

ORIUS differs in emphasis. It freezes degraded observation itself as the opening object of the runtime step. Reliability loss is not merely one more disturbance bound; it is the mechanism that tells the runtime when the meaning of observed-state legality has changed. That is why Detect and Calibrate appear before Shield. The runtime does not only ask how to project action back into a safe set. It asks how the safe set should be redefined once the observation channel can no longer be trusted at face value.

4.6 Certificate emission and operational semantics

Certify is not bookkeeping. Once repaired action is released, the runtime must still say what assumptions justified release, which fallback would be taken if the certificate expired, and

Table 4.3: Universal DC3S pipeline stages.

Stage	Name	Adapter Method	Output
1	Detect	compute_oqe	w_t, flags
2	Calibrate	build_uncertainty_set	$\mathcal{U}_t$, meta
3	Constrain	tighten_action_set	$\mathcal{A}_t$
4	Shield	repair_action	safe_action, repair_meta
5	Certify	emit_certificate	certificate

how later review can reconstruct the decision. In ORIUS, certificate metadata therefore sits inside the same runtime object as reliability, repair, and fallback.

This is what turns ORIUS from a controller-side heuristic into a governed runtime layer. The release record must be causal, bounded, and auditable. That discipline is what makes it possible to compare domains through runtime artifacts rather than after-the-fact narrative.

4.7 Runtime cost and operational realism

ORIUS is written for operational settings rather than theorem exposition alone. Runtime overhead, fallback behavior, and auditable certificates are therefore part of the scientific object. A safety layer that cannot act in time, cannot preserve a coherent lifecycle state, or cannot explain why it intervened is not a convincing architecture for physical AI.

That operational constraint is what leads naturally into the next two chapters. Once the runtime layer has been fixed as the central object, the next question is what can actually be proved about it and how those claims can be measured without confusing nominal coherence for physical safety.

4.8 Architectural takeaway

What this chapter establishes is architectural rather than empirical closure. ORIUS provides a reusable runtime contract that translates degraded observation into reliability-conditioned uncertainty, tightened action legality, repaired or fallback release, and certificate-aware governance. Stronger claims about optimality, plant realism, or deployment maturity remain tied to the benchmark rows and governed evidence surfaces that follow.

Chapter 5

Mathematical Foundations and Safety Guarantees

5.1 Why the theorem surface is intentionally narrow

ORIUS is not defended by one grand theorem claiming safety for every plant, every sensor regime, and every domain. The mathematical surface is intentionally narrower and therefore more durable. The theorem ladder states what can be proved once degraded observation is modeled explicitly, once reliability-conditioned uncertainty is admitted as part of the runtime object, and once repaired action or fallback is required before release.

That discipline matters because it keeps the dissertation from making a false choice between abstraction and implementation. The theorem surface is written against typed runtime objects, explicit assumptions, and artifact-backed diagnostics. As a result, the strongest claims are not merely conceptual and they are not merely empirical.

5.2 Notation and assumption discipline

The formal objects used in this chapter are simple but exact:

- a latent physical state x_t;
- an observed state or telemetry-derived state z_t;
- a candidate action a_t proposed by nominal intelligence;
- a repaired or fallback action $\tilde{a}_t$ released by ORIUS;
- a reliability weight $w_t \in [0, 1]$ summarizing observation trust; and

- a violation predicate $\mathcal{V}(x_t, a_t)$ defined through the domain adapter.

The dissertation does not hide its assumptions. Every strong statement depends on the assumption register in Appendix a. The core assumptions are finite observation-consistent state inflation, availability of a feasible repaired action or bounded fallback, causal certificate issuance, dual-trajectory replay fidelity, adapter faithfulness, and evidence-tier discipline. If any of those assumptions is violated, the associated theorem is not silently inherited by the runtime row.

5.3 Definition of OASG as a formal event

Definition 5.1 (Observation–Action Safety Gap)**.** An observation–action safety gap occurs at time t when the candidate action is admissible under the observed state but unsafe under the true physical state:

$$\mathcal{S}_{\text{obs}}(z_t, a_t) = 1 \quad \text{and} \quad \mathcal{V}(x_t, a_t) = 1.$$

This definition is deliberately agnostic to plant family. The adapter chooses the local meaning of admissibility and true-state violation. The universal layer fixes the semantic pattern of the failure event.

5.4 The theorem ladder

The current dissertation defends four theorem-level statements.

Theorem 5.2 (OASG existence under degraded observation)**.** *If the observation channel admits a degradation regime in which the observed-state safety predicate is no longer informationally sufficient for the true-state safety predicate, then there exists an admissible observed-state release that produces a true-state violation.*

Theorem 5.3 (Safety preservation under repaired release)**.** *Assume the inflated observation-consistent set is conservative with respect to the latent state and assume the repair or fallback operator returns only actions admissible over that set. Then the ORIUS release step preserves the defended safety predicate whenever the required repair or fallback remains feasible.*

Theorem 5.4 (Degradation-sensitive risk envelope)**.** *If uncertainty inflation is monotone in decreasing reliability and the release step is monotone in admissible-set tightening, then the risk envelope contracts as reliability falls. In other words, intervention pressure is a lawful consequence of degraded observation rather than an arbitrary heuristic.*

Proposition 5.5 (No free safety under quality-ignorant release)**.** *A controller that ignores observation-quality degradation can remain observed-state legal while incurring nonzero true-state violation under admissible degradation regimes, even when the nominal controller is internally coherent and locally optimal on the observed state.*

The point of this ladder is modest but important. ORIUS does not prove that every plant can avoid intervention. It proves that once reliability affects the observation-consistent state set, safe release must itself become reliability aware.

Table 5.1: DC3S Theorem Dependency and Proof Completeness. Each formal result is implemented in code and tested by at least one automated test.

Result	Name	Depends On	Proof Method	Code Referen
Defn. 1	OQE reliability weight w_t	—	Definition	`quality.py:co`
Prop. 1	Inflation monotonicity	Defn. 1	Analytic	`coverage_theo`
Thm. 1	Marginal coverage guarantee	Prop. 1, Defn. 1	Conformal CP theory	`coverage_theo`
Thm. 3	Expected violation bound	Thm. 1	Expectation calculus	`coverage_theo`
Thm. 9	Group-conditional coverage	Thm. 1	Mondrian CP	`coverage_theo`
Thm. 10	Hoeffding high-prob. bound	Thm. 3	Hoeffding inequality	`coverage_theo`
Thm. 11	Byzantine OQE reliability bound	Defn. 1	Trimmed-mean theory	`quality.py:co`

All results are unit-tested in `tests/test_conditional_coverage.py` and `tests/test_adversarial_faults.p`

5.5 Risk envelope interpretation

The risk envelope is the most important bridge between formal and empirical language in the dissertation. ORIUS does not claim safety from a single forecast error number. It reasons through miscoverage, intervention feasibility, and mean reliability. That framing is what allows the theorem layer to connect to uncertainty calibration without collapsing into a purely statistical dissertation.

The uncertainty references in Chapter 2 matter here. Classic conformal prediction supplies the disciplined interpretation of uncertainty sets under exchangeability, while adaptive conformal methods show how long-run calibration language can be generalized more honestly under nonstationary streams [1–3]. ORIUS does not claim that these methods alone solve runtime safety. It uses them to clarify what the uncertainty set is allowed to mean before repaired release is evaluated.

5.6 Why the proofs stay battery-backed but the objects stay universal

Battery remains the deepest theorem-to-code-to-artifact chain because the witness row provides the richest availability of proof-facing diagnostics, replay traces, and locked artifacts. That does not make the theorem objects battery-specific. The objects are written against reliability, uncertainty inflation, admissible action tightening, repair or fallback, and certificate causality. Those are universal runtime objects instantiated through the adapter, not battery-exclusive definitions.

5.7 Theorem-to-artifact linkage

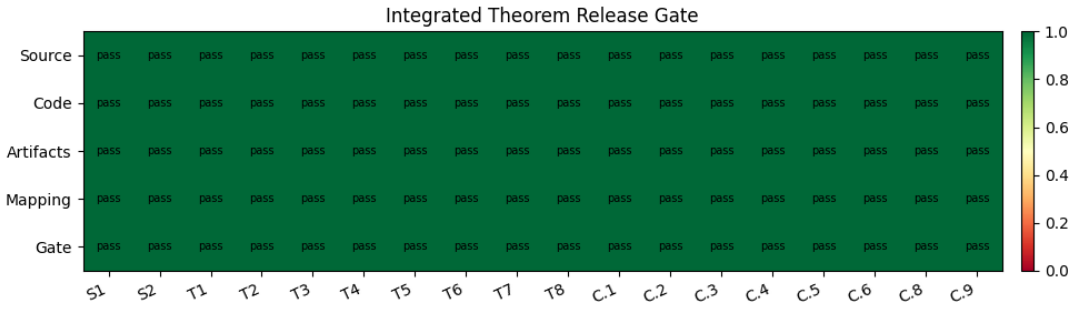

Figure 5.1: The integrated theorem gate keeps theorem language aligned to typed contracts, assumptions, and locked artifacts rather than to informal prose alone.

Figure 5.1 summarizes the discipline used throughout the manuscript. Every theorem is attached to an assumption family, a typed runtime object, and at least one artifact-facing diagnostic or replay surface. This is why the dissertation can state theorem results without pretending that empirical maturity is equal across all rows.

5.8 What is proved, validated, and still open

The theory chapter therefore ends with an explicit boundary. Theorem statements live here and in Appendix b. Validated claims only advance when replay, calibration, runtime, and governance surfaces exist for the relevant domain row. Roadmap claims remain outside defended theorem language. That separation is not a stylistic preference. It is the mechanism that lets ORIUS argue for a universal runtime safety layer without pretending that equal-domain empirical closure has already been earned.

Chapter 6

Benchmark and Reproducibility Protocol

6.1 Why ORIUS-Bench is central to the thesis

Once the runtime object has been fixed, the next question is whether it can be measured in a way that matches the theory rather than only the software stack. ORIUS-Bench is the answer to that question. It is not infrastructure attached after the science; it is the measurement contract that makes the dissertation's central object observable. If OASG is the gap between observed-state legality and true-state safety, then the benchmark must expose both surfaces explicitly. A benchmark that measures only forecast error, controller cost, or observed-state success cannot validate the ORIUS claim because it cannot see the hidden failure event the book is about.

6.2 Dual-trajectory evaluation

Every ORIUS-Bench episode therefore carries two trajectories:

- the *observed* trajectory available to the runtime after degradation; and
- the *true* trajectory used only by the replay or evaluation harness to judge whether release was physically safe.

That separation makes the benchmark stricter than a conventional controller comparison. A nominal controller may still appear coherent on the observed state while the replay harness records a true-state violation. ORIUS-Bench treats that divergence as first-class evidence.

6.3 Deterministic replay and fault schedules

The benchmark uses deterministic fault schedules so that comparisons are reproducible and so that certificate traces can be replayed under the same degraded-observation regime. Fault families include dropout, stale holds, delay, reordering, spikes, drift, and domain-local analogs. The scientific point is not merely that faults exist. It is that the same fault grammar can be replayed against one release contract across domains.

Strict replay discipline matters for another reason: it prevents the dissertation from hiding behind one-off operational anecdotes. A run must be reproducible through a manifest, fault schedule, and artifact family. If a result cannot be regenerated from the tracked surface, it is not part of the defended thesis claim.

6.4 Metrics that match the runtime object

ORIUS-Bench measures the runtime layer through seven core metrics:

TSVR true-state violation rate, the primary safety outcome;

OASG the observed-safe / true-unsafe discrepancy rate;

CVA certificate-valid action rate or validity surface;

GDQ graceful degradation quality under fallback or reduced-operation modes;

IR intervention rate, capturing how often the runtime must repair or deny release;

AC audit completeness, measuring whether release is reconstructable after the fact;

RL runtime latency, measuring whether the protection layer remains operationally usable.

This metric family matters because it keeps the book from collapsing into a single leaderboard score. ORIUS is not about RMSE alone, and it is not about controller cost alone. It is about safe release under degraded observation. The benchmark therefore measures whether release remained safe, whether intervention was required, whether the certificate remained meaningful, and whether the runtime stayed fast enough to matter.

6.5 Strict universal mode and legacy compatibility

The dissertation adopts a strict universal reading of the benchmark. Domain-neutral violation predicates, observed-state legality predicates, and constraint margins must flow through the

Table 6.1: Canonical universal replay output. Battery is the locked reference row; industrial and healthcare are proof-validated; AV is bounded proof-validated under the TTC contract; navigation is shadow-synthetic; aerospace is experimental.

Domain	Faults	Baseline TSVR	ORIUS TSVR	Reduction (%)
battery	multi	0.75	0.875	-16.7
navigation	multi	0.6667	0.25	62.5
industrial	multi	1.0	0.0	100.0
healthcare	multi	1.0	0.5833	41.7
aerospace	multi	1.0	0.0	100.0
vehicle	multi	0.0833	0.0	100.0

adapter contract. Legacy battery-shaped compatibility fields may remain in the codebase for historical or bounded-support reasons, but they are not allowed to define the universal benchmark story in the manuscript.

That distinction keeps the witness row from quietly redefining the universal object. Battery is the deepest defended row because its evidence chain is strongest, not because the benchmark is secretly battery-specific. ORIUS-Bench must therefore be written in domain-agnostic terms even when the strongest current replay surface is battery-backed.

6.6 Artifact provenance and claim locking

The benchmark protocol is inseparable from the claim-lock pipeline. The canonical thesis surface is `paper/paper.tex`, but its headline tables and figures are backed by manifests, publication matrices, and governed artifact paths. This is what prevents the manuscript from becoming a persuasive story about results that cannot later be checked.

The repository's publication surfaces already enforce this discipline. The parity matrix, submission scorecard, runtime/governance matrices, and deployment-validation scope are not mere reporting layers. They are the machine-readable surfaces that keep prose aligned with evidence status.

6.7 Reproducibility checklist

For a result to count as defended in this dissertation, the following conditions must hold:

1. the domain row must have an explicit safety predicate and adapter semantics;
2. the replay must expose true and observed trajectories;
3. the fault schedule and manifests must be reproducible;

4. the result must land in a tracked artifact family; and
5. the claim validator must allow the corresponding manuscript language.

This checklist is stricter than "the code ran once." It is the mechanism by which ORIUS turns software artifacts into scientific evidence.

6.8 Benchmark takeaway

What this chapter adds to the argument is a measurement discipline equal to the runtime object introduced in Chapters 3 and 4. ORIUS-Bench makes degraded-observation safety measurable through dual-trajectory replay, deterministic fault injection, runtime metrics matched to the release object, and claim-locked provenance. Equal-domain maturity still depends on which rows clear those requirements, but the measurement contract itself is now stable across the dissertation.

Part III

Witness Domain Evidence

Chapter 7

Witness Domain Implementation

7.1 Why battery is the witness domain

Battery remains the witness domain because it is the clearest place in the repository where the full ORIUS stack can be seen at once: degraded telemetry, reliability estimation, uncertainty inflation, repaired dispatch, fallback behavior, certificate traces, replayable fault schedules, and claim-locked artifact surfaces. That makes it the right row to demonstrate end-to-end doctrine closure without allowing the dissertation to collapse into an energy-only story.

The witness role is methodological rather than rhetorical. Battery is not the identity of ORIUS. It is the row in which the theorem layer, the implementation layer, and the artifact layer are all strong enough to support the book's deepest defended argument.

7.2 Witness-domain problem and plant semantics

The battery plant exposes a particularly clear form of OASG. Dispatch legality is computed against observed state of charge, availability, and feeder conditions, but the physical asset evolves on the true state. When telemetry becomes stale or corrupted, the optimizer can continue to return actions that appear feasible on the observed state while the physical asset is already drifting toward an unsafe or infeasible region.

That clarity is useful for a witness row because it makes visible what ORIUS actually changes: the release boundary, not merely the forecast. The runtime widens the observation-consistent state, recomputes admissible dispatch, repairs or curtails candidate action, and emits a certificate describing why release remains justified.

7.3 Data surfaces and dataset cards

The witness row is artifact-first rather than repository-inline. The battery surface is therefore described through tracked dataset cards, manifests, and publication artifacts instead of through the fiction that all raw data live inside the git tree. That design is not a weakness. It is how a dissertation remains reproducible when its empirical surface depends on large external or governed datasets.

Table 7.1: Dataset-card inventory for the conference release family.

dataset_key	dataset_label	country	rows	date_start	date_end	targets	min_coverage_pct	feature_count	columns	source	weather_source	carbon_source
DE	Germany (OPSD)	DE	17377	2018-10-07	2020-09-30	load_mw,solar_mw,wind_mw	100.000000	94	98	OPSD + SMARD	Open-Meteo archive API	SMARD generation mix
US_ERCOT	USA (EIA-930 ERCOT)	US	13453	2024-07-01	2026-01-20	load_mw,solar_mw,wind_mw	100.000000	62	62	EIA-930 ERCOT	Open-Meteo archive API	proxy_generation_mix
US_MISO	USA (EIA-930 MISO)	US	13638	2024-07-01	2026-01-20	load_mw,solar_mw,wind_mw	100.000000	114	118	EIA-930 MISO	Open-Meteo archive API	proxy_generation_mix
US_PJM	USA (EIA-930 PJM)	US	13639	2024-07-01	2026-01-20	load_mw,solar_mw,wind_mw	100.000000	62	62	EIA-930 PJM	Open-Meteo archive API	proxy_generation_mix

The dataset-card layer does more than document provenance. It tells the reader which geographies, telemetry resolutions, market structures, and operating contexts actually support the witness claim. That keeps the battery row from drifting into a false claim of energy-system universality.

7.4 Forecasting, calibration, and reliability surfaces

The witness row deliberately separates three roles that are often blurred in energy papers:

1. the forecasting surface that supports nominal dispatch;
2. the uncertainty or calibration surface that informs admissible release; and
3. the reliability surface that decides how aggressively the runtime must tighten action when telemetry degrades.

That separation matters because ORIUS is not strongest where it finds the best forecaster. It is strongest where reliability, uncertainty, repair, and certificate semantics can be shown working together on one defended row.

Taken together, the tables and figure support a bounded but important point. Even when deep forecasting does not dominate the engineered tabular baseline, learned reliability remains a scientifically relevant extension because ORIUS cares about release under degraded sensing, not only about point prediction quality.

7.5 Control baselines and adapter implementation

Battery is also the cleanest row for showing the adapter boundary in operational terms. The adapter binds:

Table 7.2: Promoted six-model forecasting comparison. Deep learning baselines evaluate point-forecast performance, confirming GBM's operational dominance in this tabular regime. Conformal uncertainty artifacts (PICP, Width) are materialized exclusively for the production-candidate model (GBM).. *— = n/a; PICP@90 and Width only for GBM.*

Region	Target	Model	RMSE	MAE	sMAPE (%)	R^2	PICP@90	Width (MW)
DE	Load	**GBM**	**305.12**	**187.87**	**0.39**	**0.9988**	88.2	1070.2
DE	Load	LSTM	—	—	—	—	—	—
DE	Load	TCN	—	—	—	—	—	—
DE	Load	N-BEATS	3646.28	2897.93	5.84	0.8347	—	—
DE	Load	TFT	8996.05	7892.52	15.52	-0.0060	—	—
DE	Load	PatchTST	4087.86	3142.77	6.22	0.7923	—	—
DE	Wind	**GBM**	**219.11**	**129.36**	**2.41**	**0.9991**	93.7	833.9
DE	Wind	LSTM	—	—	—	—	—	—
DE	Wind	TCN	—	—	—	—	—	—
DE	Wind	N-BEATS	7571.15	6232.12	67.76	-0.0504	—	—
DE	Wind	TFT	7312.89	5540.71	63.80	0.0200	—	—
DE	Wind	PatchTST	7383.74	5974.24	66.52	0.0009	—	—
DE	Solar	**GBM**	**240.35**	**117.32**	**69.24**	**0.9993**	78.8	1037.7
DE	Solar	LSTM	—	—	—	—	—	—
DE	Solar	TCN	—	—	—	—	—	—
DE	Solar	N-BEATS	3441.45	2192.56	104.80	0.8553	—	—
DE	Solar	TFT	9067.28	8041.60	128.70	-0.0042	—	—
DE	Solar	PatchTST	3136.94	2030.22	102.24	0.8798	—	—

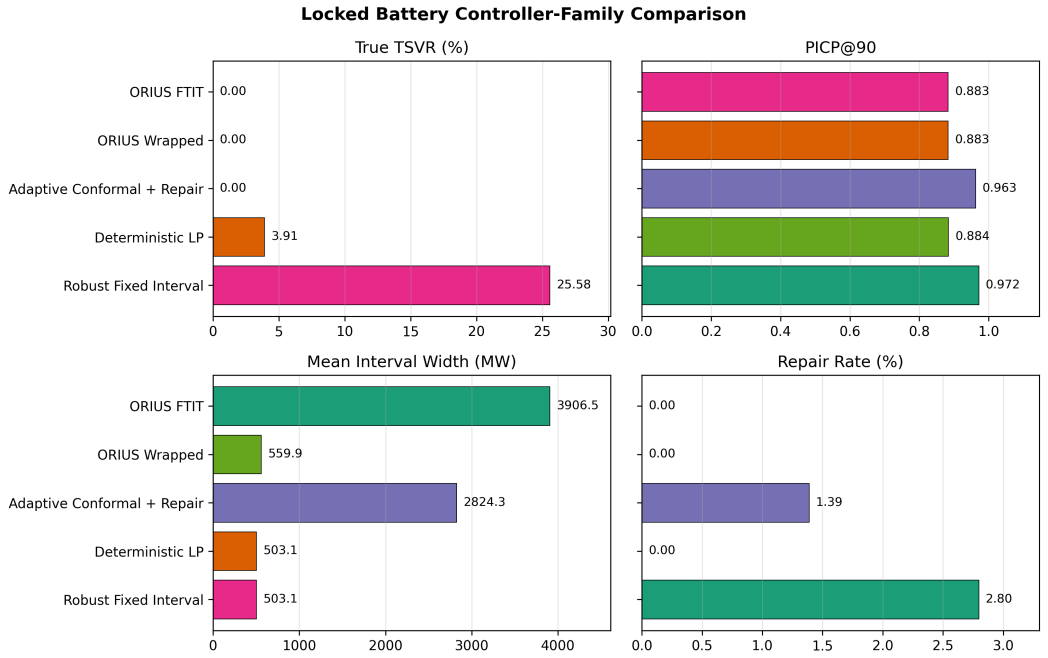

Figure 7.1: Witness-domain reliability baselines. The engineered forecasting surface and the learned reliability surface are intentionally reported together so the dissertation does not confuse forecasting novelty with degraded-observation novelty.

Table 7.3: Battery DeepOQE diagnostics on the held-out degraded-telemetry episodes.

Metric	Heuristic	DeepOQE
Reliability MAE	0.1176	0.1622
Fault AUC	—	0.5345
Low-reliability fault lift	—	0.000

- telemetry parsing and state semantics;
- the true-state safety predicate;
- dispatch feasibility and repair geometry;
- certificate payload fields; and
- the bounded fallback policy used when admissible repaired action disappears.

This is where the implementation philosophy becomes concrete. The nominal forecasting and optimization stack can vary, but the ORIUS adapter contract and replay schema remain fixed.

7.6 Locked artifact protocol

The witness row is defended through locked artifacts rather than narrative memory. Training surfaces, replay outputs, theorem traces, calibration diagnostics, deep-reliability summaries, runtime tables, and parity surfaces all point back to generated publication artifacts or manifests. This is where the chapter-first dissertation model and the repository's claim-lock machinery meet: the prose is curated, but the evidence remains machine-governed.

7.7 Witness-row scope

Battery carries the book's deepest defended claim because it closes the full runtime-safety chain from degraded observation to governed release. It does not prove that every battery chemistry, market policy, or asset-health objective is universally solved. Instead, it gives the dissertation a calibration surface: one row where theorem, runtime, and artifact closure can all be inspected together before the argument is lifted across domains.

Chapter 8

Witness Results and Failure Analysis

8.1 The governing empirical question

The witness row answers one question for the book as a whole: when telemetry degrades, can a runtime layer reduce true-state safety loss without pretending that nominal controller legality is sufficient? Battery is the deepest defended empirical surface because it closes the full ORIUS loop from degraded observation to repaired action to governed release.

That framing matters for how the results should be read. The point of the witness chapter is not to crown one battery optimizer. It is to show, on the row where the evidence chain is richest, that the release boundary behaves differently once degraded observation is treated as a first-class safety object.

8.2 Experiment design and baseline logic

The witness experiments are not designed to show that ORIUS is the best optimizer under all cost metrics. They are designed to test a narrower and more important claim: whether a certificate-aware runtime can keep hidden true-state violations from passing through a nominally coherent observation surface. The baseline set is therefore intentionally mixed. It includes stronger nominal optimizers, interval-aware baselines, and controller variants that already possess some robustness logic.

This is what gives the chapter its value. The scientific question is not simply who optimizes best in nominal conditions. It is what remains safe once the observation channel itself begins to fail.

8.3 Headline result

The main table should be read as the empirical hinge of the dissertation, because it tests the point where the formal argument meets defended runtime evidence.

Table 8.1: Main safety and cost outcomes under telemetry faults.

controller	true_soc_violation_rate_mean	true_soc_violation_rate_ci_low	true_soc_violation_rate_ci_high	true_soc_violation_severity_p95_mean	true_soc_violation_severity_p95_ci_low	true_soc_violation_severity_p95_ci_high	intervention_rate_mean	intervention_rate_ci_low	intervention_rate_ci_high	cost_delta_pct_mean
aci_conformal	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.020833	0.020833	0.020833	0.023244
cvar_interval	0.191667	0.119427	0.274305	333.582626	0.265437	848.045446	0.000000	0.000000	0.000000	-0.009092
dc3s_ftit	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.088194	0.070139	0.105556	0.147639
dc3s_wrapped	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.054576
deterministic_lp	0.129861	0.105555	0.170833	547.992608	198.317264	1056.208602	0.000000	0.000000	0.000000	0.000000
robust_fixed_interval	0.191667	0.119427	0.274305	333.582626	0.265437	848.045446	0.000000	0.000000	0.000000	-0.009092
scenario_mpc	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.006481
scenario_robust	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.006481

Several plant-specific robust baselines already avoid true-state violations on this witness task, so the contribution is not that ORIUS is the only safe controller in the comparison. The stronger point is architectural. ORIUS closes the hidden violation mode while keeping intervention, repair, fallback, and certificate semantics explicit at runtime. Deterministic and interval baselines still exhibit material true-state violation once telemetry faults are introduced, even though they remain internally coherent optimization procedures on the observed state.

That distinction is the witness-domain result. ORIUS is not replacing every robust controller with one universal optimizer. It is demonstrating that a runtime layer can sit between nominal control and physical release, detect when observation quality has changed the meaning of legality, and then repair or deny release under explicit certificate semantics.

8.4 Mechanistic interpretation

The witness row supports a mechanistic reading rather than a leaderboard reading. ORIUS changes the release decision in three linked ways:

1. reliability loss is detected before action release;
2. uncertainty is widened before legality is checked; and
3. unsafe actions are either repaired or replaced before they reach the plant.

The headline table is therefore evidence that the release boundary changed, not merely that one model forecasted more accurately than another.

8.5 Ablations and where the gain comes from

The ablation table shows where the runtime stack earns its gains and where those gains still depend on the structure of the degraded-observation regime.

Table 8.2: Ablation study with strict gate fields and statistical tests.

Scope	Fault	n	Viol. base	Viol. DC3S	Viol. red.	p_v	Sev. base	Sev. DC3S	Sev. red.	p_s	Pass
primary	all	160	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	delay	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	dropout	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	out_of_order	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary	spikes	40	0.25	0.23	8.0	0.000	0.25	0.23	8.0	0.000	no
secondary_scenario	aggregate	10	0.35	0.30	14.3	0.002	0.35	0.30	14.3	0.002	yes
secondary_scenario	aggregate	10	0.25	0.30	-20.0	0.002	0.25	0.30	-20.0	0.002	no
secondary_scenario	aggregate	10	0.35	0.30	14.3	0.002	0.35	0.30	14.3	0.002	yes
secondary_scenario	aggregate	10	0.25	0.30	-20.0	0.002	0.25	0.30	-20.0	0.002	no

This table prevents a simplistic reading of the witness result. ORIUS is not buying safety through one fixed extra margin. It behaves like a genuine degraded-observation runtime layer whose effect depends on the coupling between reliability estimation, uncertainty widening, and local repair geometry. Some scenarios clear the strict gate more comfortably than others, which is exactly what should happen when the runtime is responding to different fault structures rather than papering over them.

8.6 Conditional coverage and reliability slices

ORIUS depends on reliability-conditioned uncertainty rather than on one globally fixed interval. The grouped coverage table below shows why that distinction matters.

Table 8.3: Regime-group CQR coverage and interval width summary.

Target	Regime	PICP@90 (%)	Width (MW)	n
Load	LOW	85.2	1244.8	840
Load	MED	88.3	937.4	840
Load	HIGH	91.1	1028.2	840
Wind	LOW	97.6	719.7	840
Wind	MED	93.5	812.0	840
Wind	HIGH	90.0	969.9	840
Solar	LOW	80.2	891.6	840
Solar	MED	82.2	1016.6	845
Solar	HIGH	73.8	1206.2	835

Coverage quality moves materially across operating regimes and reliability buckets. That is precisely the pattern ORIUS is designed to take seriously. If interval behavior were uniform, a single robust margin would come closer to sufficing. Because interval behavior changes with regime and observation trust, the runtime must tighten admissible action as confidence in the observation surface falls.

8.7 Operational traces and certificate-aware behavior

The summary tables matter only because the operational traces explain how the gains are earned. Intervention rate, fallback usage, and certificate continuity show whether ORIUS is improving safety through disciplined runtime mediation or merely through indiscriminate shutdown. That distinction is essential. A controller can always buy apparent safety by refusing to act. The witness row matters because ORIUS reduces hidden safety loss while keeping intervention bounded, reviewable, and operationally usable.

8.8 Bounded deep-learning novelty inside the witness row

The witness row also contains a bounded learned-reliability novelty surface. It matters here only insofar as it improves the degraded-observation safety argument rather than turning the dissertation into a forecasting paper.

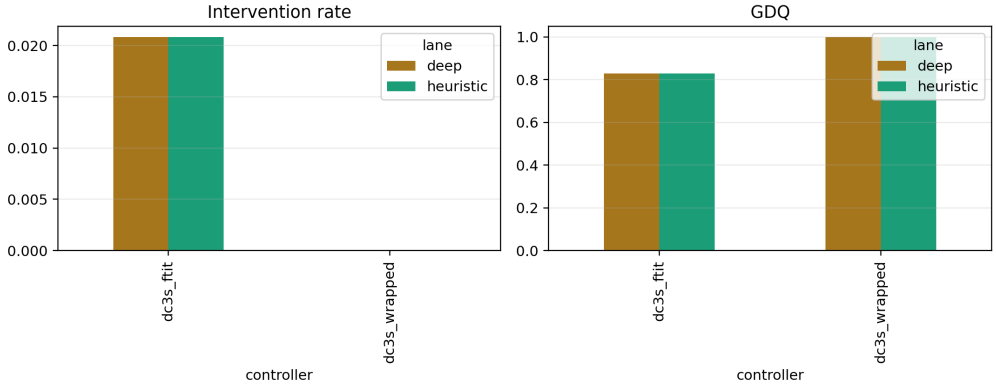

Figure 8.1: Battery learned-reliability safety surface. The deep model is judged by true-state safety, intervention burden, and certificate-valid release behavior rather than by forecast error alone.

The figure supports a narrower point than the main safety table. It shows that the learned reliability path can improve the runtime's discrimination of when to intervene and when to allow nominal action to pass. That is a bounded novelty claim about the observation side of ORIUS, not a replacement for the main witness claim about repaired release under degraded telemetry.

8.9 What fails without the runtime layer

Without ORIUS, the telemetry fault is treated as a nuisance external to the release decision. The deterministic and interval baselines show what follows from that assumption: the controller remains computationally coherent while true-state violation reappears. Without calibrated widening, legality is checked against the wrong state set. Without repair, unsafe action is still released. Without certificate discipline, the runtime cannot later explain why the release was allowed or when fallback should have taken over.

8.10 Witness-row claim boundary

The witness chapter supports three claims and no more. First, ORIUS can materially improve degraded-observation safety on a defended row. Second, the kernel remains meaningful across multiple telemetry fault families rather than on one curated failure. Third, intervention and certificate behavior are part of the scientific result, not merely surrounding instrumentation. It does not claim that the battery row proves equal-domain closure or settles every question about deployment economics, asset aging, or market-optimal dispatch. What it does show is that once observed-state legality is no longer enough, ORIUS changes the release boundary in a measurable, reviewable, and auditable way.

Part IV

Universalization and Extensions

Chapter 9

Universal ORIUS Across Domains

9.1 What universality means in this dissertation

ORIUS is universal in a specific and intentionally disciplined sense. It does not assume that batteries, vehicles, industrial plants, healthcare systems, navigation stacks, and aerospace controllers share a plant model, control law, or optimization target. It assumes only that each domain must answer the same release-boundary question once observation becomes unreliable: what action may still be released safely, under what margin, with what fallback, and under what certificate?

The shared object is therefore not the nominal controller. It is the runtime safety logic that mediates degraded observation before actuation. This is why the monograph can make a strong universality claim without flattening domain structure.

9.2 A transfer recipe rather than a portability slogan

The dissertation treats domain transfer as a recipe, not a slogan. A new ORIUS row is only meaningful if it specifies:

1. the local degraded-observation hazard;
2. the true-state violation predicate and observed-state legality predicate;
3. the action-repair or fallback geometry;
4. the certificate payload and lifecycle semantics; and
5. the replay and evidence surface through which that row will be judged.

That transfer as a recipe is what converts structural universality into a scientific program. Without it, "universal" would reduce to analogy. With it, the reader can inspect exactly how a domain is instantiated and where its evidence status currently sits.

9.3 Cross-domain status at a glance

Table 9.1: Equal-domain parity gate for the universal-first ORIUS book.

Domain	Adapter	Raw/Data	Train	Replay	Soundness	Fallback	CertOS	P5	Tier / Exact blocker
Battery Energy Storage	pass	locked_reference	pass	pass	reference_witness	paper6_runtime	evaluated	evaluated	reference; battery_reference_witness
Autonomous Vehicles	pass	verified	pass	pass	pass	bounded_runtime_pass	evaluated	gated	proof_validated; promoted
Industrial Process Control	pass	verified	pass	pass	pass	bounded_runtime_pass	evaluated	evaluated	proof_validated; industrial_train_validation_chain_complete
Medical and Healthcare Monitoring	pass	verified	pass	pass	pass	bounded_runtime_pass	evaluated	gated	proof_validated; healthcare_train_validation_chain_complete
Navigation and Guidance	pass	blocked	blocked_real_data_gap	pass	blocked_real_data_gap	bounded_runtime_pass	gated	gated	shadow_synthetic; navigation_real_data_gap
Aerospace Control	pass	placeholder_synthetic	placeholder_surface	pass	experimental_placeholder	bounded_runtime_pass	gated	gated	experimental; aerospace_experimental_placeholder

The parity matrix belongs in the main line of the argument because it keeps the universality claim honest. Battery is the witness row. Industrial process control and healthcare monitoring are defended bounded rows. The autonomous-vehicle row is a defended bounded runtime row under the current artifact surface. Navigation remains blocked by the missing real-data closure row, and aerospace remains blocked by the absence of a defended multi-flight runtime surface with material post-repair gain.

9.4 Domain synthesis

The point of the synthesis is not to suggest that every row is interchangeable. It is to show how the same release-boundary grammar survives contact with different plants, action spaces, and safety objects while each row keeps its own evidence ceiling.

Battery energy storage. The battery hazard is hidden constraint violation under degraded feeder, load, or state telemetry. ORIUS instantiates the runtime with tight state margins, explicit repair, and auditable release over a heavily instrumented control surface. Its universality contribution is to show that the full theorem-to-code-to-artifact chain can be closed on a real physical control problem.

Autonomous vehicles. The autonomous-vehicle hazard is unsafe longitudinal release under stale or degraded scene understanding. ORIUS instantiates the runtime through bounded time-to-collision and predictive-entry-barrier logic, with explicit brake-oriented

fallback and certificate-aware release checks. Its universality contribution is to show that the same release semantics remain meaningful in a fast-horizon, continuous-action mobility setting.

Industrial process control. The industrial hazard is constraint excursion under corrupted or delayed plant telemetry. ORIUS instantiates the runtime through bounded power or process envelopes, repair against shared operating constraints, and governed fallback. Its universality contribution is to show that ORIUS can operate in tightly regulated process loops where intervention must remain operationally interpretable.

Healthcare monitoring. The healthcare hazard is clinically unsafe release under partially observed physiological state, especially when alerts and interventions are triggered from noisy or degraded streams. ORIUS instantiates the runtime through physiological margin checks, alert fallback, and certificate-aware release semantics. Its universality contribution is to show that the same runtime grammar survives in a setting where the protected object is patient-facing monitoring behavior rather than mechanical actuation.

Navigation and guidance. The navigation hazard is trajectory release under degraded localization or scene reference. ORIUS instantiates the runtime through hold-position or corridor-preserving fallback logic. Its current evidence status remains gated because the defended real-data train–replay chain is not yet closed. Its universality contribution is still scientifically useful: it shows how the ORIUS release boundary maps onto spatial feasibility problems even before the row is fully defended.

Aerospace control. The aerospace hazard is unsafe envelope or guidance release under degraded flight observation. ORIUS instantiates the runtime through envelope-aware repair, bounded fallback, and certificate semantics tied to flight safety objects. Its current evidence status remains experimental because a defended multi-flight runtime validation surface is still missing. Its universality contribution is to show that ORIUS extends to high-consequence control domains without pretending that structural portability already implies empirical closure.

9.5 Architecture universality versus parity universality

Two conclusions follow from the domain synthesis. First, what transfers is not a controller family but a release-boundary grammar: reliability loss, uncertainty inflation, action repair,

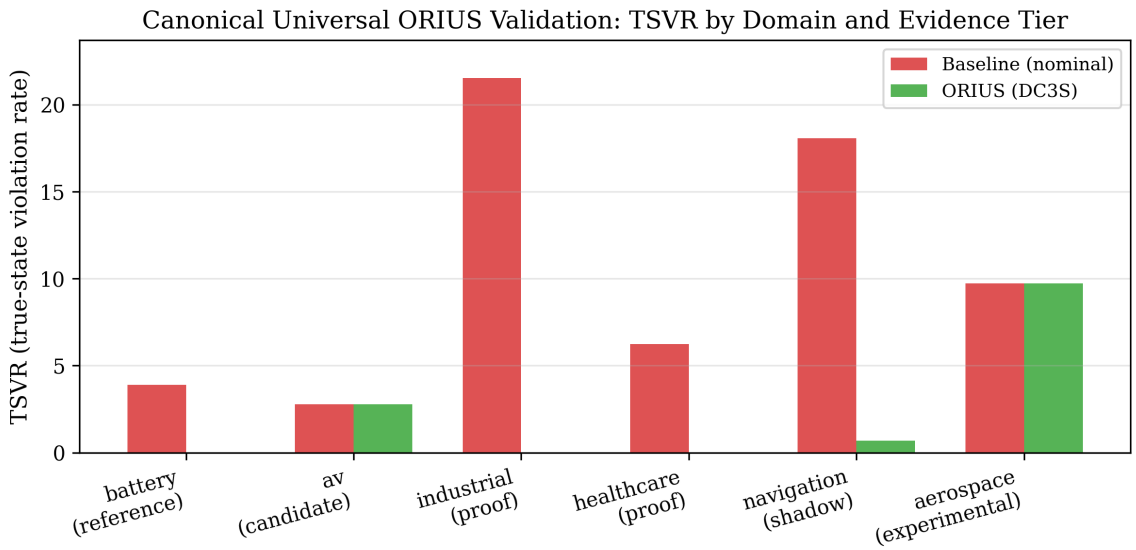

Figure 9.1: Cross-domain ORIUS validation surface. One runtime grammar carries across six domains, while each row remains limited by its own defended evidence surface.

fallback semantics, and certificate-aware release. Second, what remains local is the geometry of the safety object itself: a battery margin is not a vehicle headway constraint, and a vehicle headway constraint is not a physiological alert threshold or a flight envelope.

That is why the book distinguishes architecture universality from parity universality. Architecture universality is already defended by the shared adapter contract, kernel, benchmark schema, and governance surfaces. Parity universality is an empirical achievement that requires every row to clear the same replay, calibration, runtime, and governance gates.

9.6 Default proof mode for the dissertation

The default proof mode of the dissertation is therefore: *universal in architecture, tiered in evidence*. Battery calibrates the deepest witness depth. Industrial control, healthcare, and the bounded autonomous-vehicle runtime row show that the same contract is not confined to one plant family. Navigation and aerospace remain explicit open rows rather than rhetorical symmetry pieces.

9.7 Boundary of the universality claim

This chapter is the place where the thesis must be most explicit. The universal claim is a claim about a reusable runtime grammar, not a claim that every supported row has already reached equal maturity. The parity matrix makes that difference visible in one artifact surface.

Navigation remains blocked by the missing real-data closure row. Aerospace remains blocked by the absence of a defended multi-flight runtime validation surface with material post-repair gain. In that exact sense, ORIUS already establishes a reusable runtime safety grammar across domains, while equal-domain closure remains an empirical program that must be earned.

Chapter 10

Temporal Certificates and Graceful Degradation

10.1 Certificate horizon as a runtime object

ORIUS certificates are temporal objects, not one-step labels. A certificate has a validity horizon, an expiration condition, and a bounded interpretation under worsening observation. This matters because degraded telemetry can remain acceptable for one step and become unacceptable only a short time later. The temporal question is therefore not "safe or unsafe forever," but "how long does the runtime have before the current certificate no longer justifies release?"

10.2 Why temporal safety belongs inside runtime assurance

This temporal view is one of the places where ORIUS aligns most clearly with classical runtime-assurance intuition. Simplex-style architectures distinguish between high- performance nominal control and a safer supervisory authority that must take over when the nominal contract is no longer justified [4, 5]. ORIUS extends that idea by attaching horizon semantics directly to the certificate object. The runtime is not only deciding whether to intervene. It is also deciding how long the present release claim is allowed to remain live before fallback becomes mandatory.

10.3 Expiration, half-life, and safe-hold

The current evidence supports a conservative temporal reading. Horizon and half-life are most strongly defended as bounded expiration and safe-hold semantics under blackout-style degradation. That is the right level of claim: certificate time is measurable and governable, but it is not yet promoted into a universal law of every plant and sensor regime.

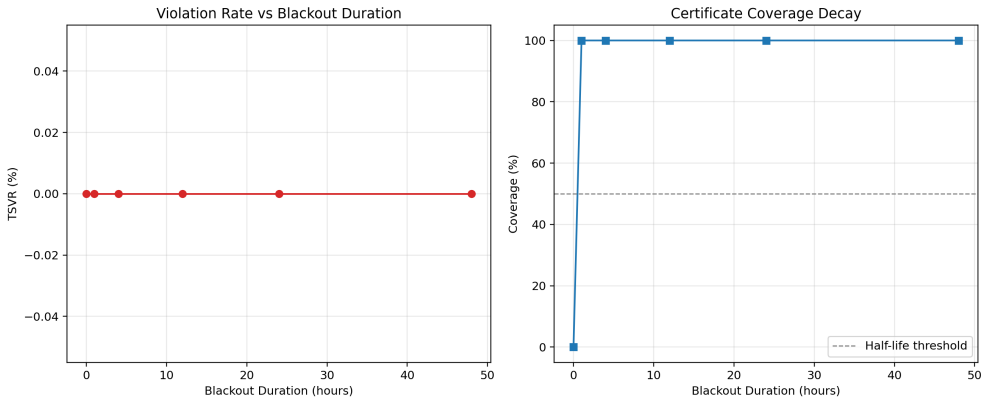

Figure 10.1: Blackout half-life surface used to motivate certificate expiration, bounded safe-hold, and temporal intervention discipline.

The half-life framing matters because it gives the runtime a language for degradation that is stronger than an unstructured timeout. A certificate can decay in justification even when the nominal policy remains capable of producing an action. ORIUS uses that decay to decide when safe-hold, curtailment, or reduced-operation behavior should replace ordinary release.

10.4 Severe-blackout protocol

The severe-blackout setting is the cleanest empirical test of temporal semantics because the runtime cannot rely on fresh observation arriving in time to rescue nominal control. The question becomes: can the runtime preserve a governed reduced-operation mode long enough to avoid unsafe release while still remaining operationally legible? That is why the blackout protocol belongs in the main body. It is not a corner-case stress test. It is the clearest demonstration that certificates are temporal objects with behavioral consequences.

10.5 Graceful degradation policies

When no full-quality control action remains justified, ORIUS benchmarks degradation policies explicitly rather than treating them as informal emergency behavior. This is the runtime-

assurance connection in practice: the system needs a governed way to do less, safer, and more explainably when observation quality collapses.

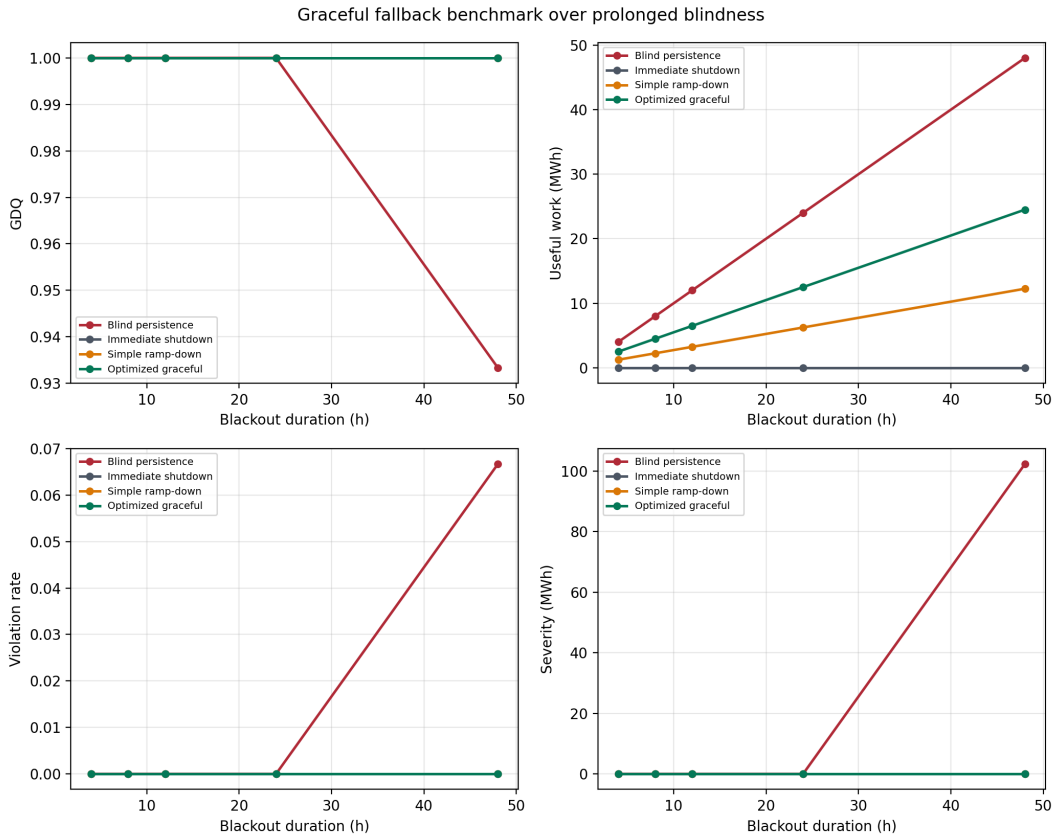

Figure 10.2: Four-policy graceful degradation comparison. The dissertation uses this family to treat fallback quality as a benchmarked layer instead of an unexamined emergency default.

Benchmarking degradation policies matters scientifically because a runtime could always buy safety by immediately collapsing to a maximally conservative shutdown or no-op. ORIUS asks a more demanding question: what bounded policy still preserves safety while keeping the system operationally interpretable and certificate-governed?

10.6 Temporal extension

This chapter extends the same runtime object introduced earlier rather than adding a separate theory of fallback. Certificate time and graceful degradation become measurable and governable parts of the release object itself. The defended claim remains bounded: time-to-expiration, safe-hold, and fallback quality now belong to the scientific evaluation surface of ORIUS, but no universal optimal degradation policy is claimed here.

Chapter 11

Compositional Safety

11.1 Why local safety does not automatically compose

One of the easiest mistakes in safety argumentation is to assume that individually safe controllers remain jointly safe when they share resources or constraints. ORIUS treats composition as a bounded extension precisely because this implication fails in general. Local certificates can remain valid while shared budgets, timing windows, or coupled plant dynamics still create system-level failure.

11.2 A simple counterexample

Consider two locally admissible controllers that each satisfy their own state and action constraints. If both consume a shared feeder budget, corridor occupancy, or actuator authority reserve, their simultaneous release can still violate the system-level constraint even though neither local controller is individually unsafe. This is the compositional version of OASG: local legality can diverge from system-level safety when the shared constraint is not represented in the local release object.

11.3 Shared constraints as the right composition surface

Composition is therefore framed around shared constraints rather than around an immediate claim of universal multi-agent closure. Shared power budgets, corridor occupancy, common actuator authority, or coupled timing envelopes are the right first-class objects because they say where local certificates stop being sufficient. ORIUS can then lift the certificate object

into a negotiated setting without pretending that single-agent repair theorems automatically solve the multi-agent case.

Table 11.1: Multi-Agent Fleet Coordination: joint safety violations and useful work under shared transformer capacity constraint (limit = 80 MW, two batteries at 60 MW each = 120 MW proposed). Independent protocol violates; ORIUS-coordinated eliminates violations.

Protocol	Joint Violations	Local Violations	Useful Work (MWh)	Margin Quality
Independent (non-compositional)	24	0	1920	0
Centralized (ORIUS-coordinated)	**0**	0	1920	1
Distributed Negotiation	**0**	0	1920	1

11.4 Multi-agent certificates and negotiation

The composition story in ORIUS is therefore narrow but meaningful. A local certificate says that an action is safe given one agent's uncertainty set and fallback semantics. A compositional certificate must additionally encode the shared constraint on which multiple agents depend. This turns negotiation and fairness from policy afterthoughts into runtime objects. If the shared constraint cannot be satisfied jointly, the runtime must decide which agents may continue, which must degrade, and which fallbacks remain legitimate.

11.5 Why composition remains bounded in the dissertation

Composition matters scientifically because it shows that the ORIUS grammar does not end at single-agent control. At the same time, it remains outside the strongest universal claim. The present dissertation does not defend an equal-domain composition theorem across all six rows. It shows that shared constraints can be represented through the same runtime object and that local certificates alone are not sufficient once coupling enters the system.

11.6 Compositional takeaway

What this chapter contributes is a bounded extension, not a new center of gravity for the thesis. ORIUS can lift degraded-observation safety into a shared-constraint setting by making system-level resources explicit in the release object. It does not claim that local safety composes automatically, and it does not claim that every current domain row already clears a defended multi-agent parity gate.

Chapter 12

CertOS Runtime Assurance

12.1 Why CertOS belongs in the main argument

CertOS is the part of ORIUS that turns repaired action into a governed runtime object. Without lifecycle state, fallback rules, and an audit chain, certificates would be inert metadata. With CertOS, certificate validity becomes part of the release decision itself: which action was allowed, under which reliability state, for which horizon, and under which fallback if the certificate expires or becomes invalid?

This chapter therefore does not read as a deployment appendix. It is the point at which the ORIUS kernel acquires operational semantics. `DomainAdapter` defines the local safety predicate and fallback object that CertOS must protect, and the universal benchmark schema defines the replay traces from which certificate validity, latency, and lifecycle behavior are evaluated.

12.2 Lifecycle operations

CertOS organizes runtime release through a small but strict lifecycle:

- issue a certificate only after the repaired or fallback action is shown admissible;
- validate a certificate before each protected release;
- renew a certificate when fresh evidence preserves the release claim;
- expire or revoke a certificate when the underlying assumptions no longer hold; and
- force explicit fallback when no valid certificate remains.

These operations make ORIUS operationally legible. The runtime is not merely deciding whether an action is numerically acceptable; it is managing the validity of the claim under which release occurs.

12.3 Runtime invariants

The lifecycle logic is narrow and strong. A protected action is never released without a valid certificate or an explicit fallback. Certificate state must be renewable, expirable, revocable, and auditable. Runtime traces must preserve enough information to reconstruct why release was allowed and when the governing assumptions ceased to hold. These are not administrative conveniences. They are the invariants that keep repaired action tied to a reviewable safety argument.

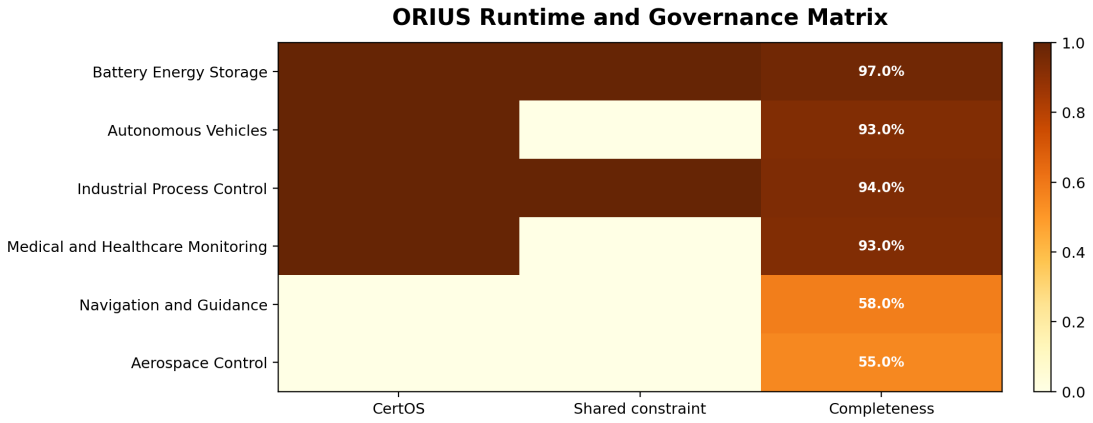

Figure 12.1: CertOS lifecycle and runtime-governance surface across ORIUS domains. The figure shows how certificate issue, validation, expiry, fallback, and audit continuity are exposed as part of runtime safety rather than as post hoc reporting.

12.4 Audit chain and post hoc verification

The audit chain matters because ORIUS is not only a forward-looking safety layer. It is also a mechanism for reconstructing the justification of past release decisions. Once an unsafe event, intervention, or fallback occurs, the system must be able to explain what the reliability state was, which repair or fallback was chosen, what certificate was valid, and why the runtime believed the action remained admissible. This is why audit completeness is a first-class benchmark metric rather than a logging implementation detail.

12.5 Governance alignment, not compliance

CertOS aligns the runtime evidence surface with disciplined safety-argument practice without claiming compliance with external standards. In that limited and precise sense, the governance chapter is legible to readers who care about claim-based safety arguments, lifecycle discipline, and trustworthy AI governance [9–12].

The distinction matters. UL 4600, the NIST AI RMF, ISO 26262, and IEC 61508 are used here as evidence-structure and lifecycle analogs only. They motivate why a runtime safety layer should make claims, assumptions, evidence, lifecycle transitions, and audit continuity explicit. They do not turn the dissertation into a compliance claim, and the manuscript must never be read that way.

12.6 Universality and evidence-tiered maturity

CertOS is universal at the level of release semantics. Every defended ORIUS row needs a certificate object, fallback policy, and audit trace. What is not yet universal is equal deployment maturity across every domain. Battery, industrial process control, and healthcare provide the strongest defended lifecycle surfaces. The autonomous-vehicle row shows a defended bounded runtime surface under current artifacts. Navigation and aerospace remain gated for the same empirical reasons already stated elsewhere in the dissertation. CertOS therefore supplies one common governance object without pretending that all deployment surfaces are equally mature today.

12.7 Governance boundary

This chapter is one of the places where the non-claim boundary has to remain explicit. ORIUS provides a policy-driven runtime governance layer in which repaired action, fallback, and certificate validity are coupled, auditable, and recoverable after the fact. That claim is scientifically important because a runtime safety layer without release semantics cannot say what action was justified once degraded observation has already entered the control loop. It does not claim formal certification compliance, and it does not claim equal deployment maturity across every current domain row.

Part V

Submission Boundary

Chapter 13

Governance, Reproducibility, Limitations, and Conclusion

13.1 What is proved

The dissertation proves a bounded but durable theory claim. Under explicit assumptions, degraded observation can create a gap between observed-state legality and true-state safety; reliability-conditioned uncertainty inflation changes the admissible release set; and repair-or-fallback discipline preserves safety whenever the inflated set and lifecycle semantics remain feasible. Those results live in the theorem chapter and proof appendix and should not be overstated beyond their assumptions.

13.2 What is empirically validated

The empirical surface is strongest at the witness and defended bounded rows. Battery remains the deepest theorem-to-code-to-artifact chain. Industrial process control and healthcare monitoring support defended bounded interpretations under the current replay and governance surfaces. The autonomous-vehicle row is a defended bounded runtime row under the present artifact surface. Navigation remains blocked by the absence of a defended real-data train–replay chain. Aerospace remains blocked by the absence of a defended multi-flight runtime validation surface with material post-repair gain.

Those limits matter because ORIUS is not presented here as a universal optimal controller or as a finished equal-domain deployment program. It is presented as a runtime safety layer whose strongest current evidence is deep in one witness domain and bounded across several others. The open rows should therefore be read as the next empirical work required to extend

the thesis, not as defects that invalidate the defended core.

13.3 What is out of scope

The dissertation does not claim:

- equal-domain empirical closure across all six domains;
- universal statistical calibration under arbitrary shift;
- production deployment readiness in every plant family; or
- formal certification compliance under UL 4600, NIST AI RMF, ISO 26262, or IEC 61508.

The standards language used in the governance chapter is an evidence-structure alignment and lifecycle-analogy argument only. The deployment language is deliberately bounded by the parity matrix, runtime/governance surfaces, and explicit non-claim statements.

13.4 Reproducibility discipline

Every headline claim stays tied to tracked figures, tables, manifests, or claim matrices so that the defended surface remains auditable after the fact. The purpose of that discipline is simple: a universal safety thesis should be inspectable at the level of artifacts, not only persuasive at the level of prose.

That is why the dissertation remains chapter-first in prose but artifact-first in evidence. The prose chapters explain the argument. The claim validator, publication matrices, release manifests, and replay tables control what the prose is allowed to say. In that exact sense, the dissertation is universal in architecture, tiered in evidence.

13.5 Threats to validity

The largest threats to validity are asymmetry and interpretation. Asymmetry arises because the witness row is much deeper than the currently blocked rows. Interpretation risk arises because readers may confuse structural universality with equal-domain parity. The dissertation addresses the first risk by keeping the witness role explicit and by preserving the parity matrix in the main narrative. It addresses the second risk by separating proved, validated, and roadmap claims at the major boundary chapters.

13.6 Conclusion

The argument of this dissertation can now be stated plainly. Degraded observation is not a peripheral nuisance attached to otherwise reliable autonomy; it is a release-boundary safety problem in its own right. Once observed-state legality and true-state legality can diverge, safe actuation requires more than nominal intelligence. It requires explicit reliability estimation, conservative state widening, repaired action or bounded fallback, and certificate-aware release.

ORIUS makes that claim operational through three connected surfaces. A `DomainAdapter` exposes the local safety predicate and fallback object. The universal benchmark schema exposes one replay contract for truth, observation, intervention, fallback, certificate validity, and latency. CertOS supplies the lifecycle semantics that determine when repaired action may still be released. Together, those surfaces turn degraded observation from a vague systems concern into a measurable and governable runtime object.

That is the durable contribution of the thesis. ORIUS does not ask every domain to share a controller, plant model, or optimization objective. It asks every domain to make the release boundary explicit. In doing so, it turns true-state violation, intervention burden, fallback usage, certificate validity, and runtime latency into scientific objects that can be compared across domains without pretending the domains themselves are identical.

The final thesis claim is therefore ambitious and disciplined at the same time. ORIUS already supports a reusable runtime safety layer for physical AI under degraded observation. It does not yet support the stronger claim that every supported domain is equally mature. In that exact sense, structural universality is defended now, while equal-domain closure remains an empirical program that must still be earned. That boundary is not a rhetorical retreat. It is part of what makes the argument credible.

Appendix a

Assumption Register

This appendix records the assumptions that delimit the dissertation's strongest claims. The purpose is not to hide caveats in the back matter. It is to make the theorem and evidence boundary explicit enough that every promoted claim can be traced to the assumptions it needs, the diagnostics that check those assumptions, and the fallback interpretation used when the assumptions become only partially satisfied.

Table a.1: ORIUS dissertation assumption register.

ID	Name	Meaning	Diagnostics / evidence	Where it matters
A1	Bounded observation error surface	Reliability-conditioned uncertainty inflation produces a finite observation-consistent state set.	Reliability summaries, calibration diagnostics, and bounded widening logic in the theorem gate.	Chapters 3, 5, and Appendix c.
A2	Feasible safe repair or fallback	When the admissible set collapses, the runtime can still choose a certified fallback action or safe-hold policy.	Repair traces, fallback tables, and CertOS lifecycle surfaces.	Chapters 4, 10, and 12.

ID	Name	Meaning	Diagnostics / evidence	Where it matters
A3	Causal certificate issuance	Certificates depend only on information available at runtime up to the current step.	Certificate payload inspection, audit chain reconstruction, and lifecycle verification.	Chapters 4, 12, and Appendix e.
A4	Dual-trajectory replay fidelity	The benchmark exposes both observed and true safety surfaces strongly enough to measure OASG.	Replay manifests, deterministic fault schedules, and domain closure artifacts.	Chapters 6, 8, and 9.
A5	Adapter faithfulness	Each domain adapter encodes the relevant safety predicate, fallback semantics, and certificate payload honestly for its plant.	Domain closure matrix, protocol-card review surfaces, and non-claim boundaries for weaker rows.	Chapters 4, 9, and 12.
A6	Evidence-tier discipline	Architectural universality and empirical parity are not allowed to collapse into one claim.	Parity matrix, submission scorecard, gap matrix, and claim validator.	Chapters 1, 9, and 13.

The dissertation's formal claims should always be read together with this table. When the evidence surface is weaker than the architectural surface, the text is intentionally written as a bounded semantic or governance statement rather than as an equal-domain validation claim.

Appendix b

Proofs

This appendix records the proof-facing structure of the dissertation. It is not intended to replace the theorem chapter; it is intended to make the proof obligations inspectable and to show which claims are formal, which are empirical, and which rely on shared typed runtime objects.

b.1 Proof map

Table b.1: Theorem-to-proof map for the ORIUS dissertation.

Result	**Main idea**	**Depends on**	**Artifact-facing link**
Theorem 5.2	If observed legality is not informationally sufficient for true-state safety, then there exists a degraded-observation regime that admits an observed-safe but true-unsafe release.	A1, A4, A5	Dual-trajectory replay and the OASG metric family.
Theorem 5.3	If inflation is conservative and the repair or fallback operator respects the tightened set, the released action preserves the defended safety predicate.	A1, A2, A3, A5	Repair traces, fallback logs, and certificate-valid release surfaces.

Result	Main idea	Depends on	Artifact-facing link
Theorem 5.4	As reliability falls, uncertainty inflation and admissible-set tightening contract the release envelope lawfully rather than heuristically.	A1, A3, A6	Calibration diagnostics, theorem gate figure, and runtime intervention traces.
Proposition 5.5	Quality-ignorant release can remain observed-safe while incurring true-state failure.	A1, A4, A5	Witness-row violation tables and fault-family replay.

b.2 Proof sketches

OASG existence. The proof proceeds by separating observed-state legality from true-state legality. Once the adapter exposes a true-state violation predicate and the replay harness exposes an observed state that is not informationally sufficient for that predicate, one can construct a degradation regime in which observed legality survives while true-state safety fails. The scientific role of the theorem is not to characterize every such regime exhaustively. It is to show that the gap is structural rather than anecdotal.

Safety preservation under repair. The second theorem uses the conservativeness of the inflated observation-consistent set. If the repair operator only emits actions admissible over that set, or the fallback operator replaces release when such action does not exist, then the released action remains safe for the latent state whenever the inflation really upper-bounds that state. This is the exact point at which fallback becomes part of the proof surface rather than an implementation afterthought.

Degradation-sensitive risk envelope. The third theorem formalizes the intuition that reduced trust must shrink release freedom. Monotone reliability loss implies monotone widening of the uncertainty set; that widened set then implies monotone tightening of admissible release. The theorem does not say intervention is free. It says intervention pressure is a lawful consequence of degraded observation.

No-free-safety proposition. The proposition follows by contradiction against any controller that checks legality only on observed state while ignoring observation degradation. If the degraded observation regime is admissible yet insufficiently informative about the latent state, then there exists a release that remains locally coherent while still violating the true safety predicate.

b.3 Interpretation

These proofs are deliberately modest in scope. They do not prove equal-domain closure and they do not replace the need for replay, calibration, runtime, and governance artifacts. Their role is to anchor the manuscript's strongest claims to explicit assumptions and typed objects so that later empirical chapters can be read as validation of the same runtime contract rather than as isolated case studies.

Appendix c

Benchmark Specifications

This appendix records the benchmark-facing specifications that make the dissertation replayable and auditable. ORIUS-Bench is universal at the schema level: every episode tracks candidate action, repaired action, true-state violation, observed-state satisfaction, certificate state, fallback or intervention markers, and runtime cost. The current deepest empirical instantiation remains the battery witness row, so the detailed fault-family table below is still battery-grounded. That grounding is deliberate: it shows the strongest defended fault surface while the benchmark contract itself remains domain-neutral.

Any remaining battery-shaped compatibility fields in the codebase should therefore be read as legacy support surfaces, not as the dissertation's definition of the benchmark. The canonical benchmark language is the dual-trajectory, adapter-driven contract described in Chapter 6.

c.1 Canonical fault families

Table c.1 defines each fault family, its configurable parameters, the tested range, and the battery-domain interpretation.

Table c.1: Witness-domain fault families used to instantiate the universal ORIUS-Bench telemetry-degradation contract.

Fault	**Parameters**	**Range tested**	**Battery interpretation**
`dropout`	drop_rate d	$d \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$	Packet loss on the BMS→EMS telemetry link. Dropped readings are replaced by the last received value (zero-order hold).
`delay_jitter`	base delay τ_b (s), jitter σ_τ (s)	$\tau_b \in \{1, 3, 5, 10\}$	Variable network latency between BMS and the dispatch engine. Delayed packets arrive out of order and are consumed at their arrival time.
`spike`	magnitude μ_s (multiplier)	$\mu_s \in \{2, 3, 4, 5\}$	Transient sensor glitch producing a reading μ_s times the true value. Models ADC saturation or electromagnetic interference on current/voltage sensors.
`stale`	hold duration h (steps)	$h \in \{2, 4, 6, 8, 12\}$	Sensor freeze: the telemetry value is held constant for h consecutive steps. Models BMS firmware hang or communication bus lock-up.
`drift`	drift rate β (per step)	$\beta \in \{0.01, 0.02, 0.03, 0.05\}$	Slow, linear bias accumulating as $\xi_t = \beta\, t$. Models sensor calibration drift (e.g. current-sensor offset) or gradual SOH-induced SOC bias.
`reorder`	buffer size B (steps)	$B \in \{2, 4, 6, 8\}$	Out-of-order packet delivery: readings within a sliding window of B steps are randomly permuted. Models multi-path routing or priority-queue inversion on industrial networks.

c.2 Combined profile: `drift_combo`

The `drift_combo` profile activates all six fault families simultaneously at their mid-range parameter values with a configurable dropout rate d. This is the default stress profile used in the main results of the witness-domain chapter and in the bounded controller-comparison surfaces that accompany the ORIUS publication artifacts.

Table c.2: `drift_combo` default parameter values.

Fault	**Parameter**	**Default value**
dropout	d	0.20
delay_jitter	τ_b	3 s
spike	μ_s	3×
stale	h	4 steps
drift	β	0.02 /step
reorder	B	4 steps

c.3 Replay and fault-injection implementation

Faults are injected by the CPSBench runner (`src/orius/cpsbench_iot/runner.py`) between the plant's true-state output and the controller's observation input. The injection pipeline:

1. **Plant emits** clean telemetry z_t.
2. **Fault layer** transforms $z_t \to \tilde{z}_t$ according to the active fault families and their parameters.
3. **Controller receives** degraded telemetry $\tilde{z}_t$ as its observation o_t.
4. **DC3S OQE** computes w_t from $\tilde{z}_t$ (detecting the degradation).

Fault parameters are set in `configs/dc3s.yaml` under the `fault_profile` key. Each evaluation run logs the active profile and parameter snapshot in the certificate metadata for reproducibility.

c.4 Severity mapping

Table c.3 maps each fault family to its primary impact on DC3S pipeline stages.

Table c.3: Fault-to-pipeline severity mapping.

Fault	**OQE** w_t	**RUI** m_t	**Shield IR**	**Cert. validity**
dropout	$\downarrow\downarrow$	$\uparrow\uparrow$	$\uparrow$	$\downarrow$
delay_jitter	$\downarrow$	$\uparrow$	$\uparrow$	$\downarrow$
spike	$\downarrow\downarrow$	$\uparrow\uparrow$	$\uparrow\uparrow$	$\downarrow\downarrow$
stale	$\downarrow$	$\uparrow$	$\uparrow$	$\downarrow$
drift	$\downarrow\downarrow$	$\uparrow\uparrow$	$\uparrow$	$\downarrow\downarrow$
reorder	$\downarrow$	$\uparrow$	$\uparrow$	—

$\downarrow$ = mild decrease; $\downarrow\downarrow$ = strong decrease; $\uparrow$ = mild increase; $\uparrow\uparrow$ = strong increase; — = negligible effect.

Appendix d

Artifact and Claim Index

This appendix records the canonical bridge from dissertation claims to governed repository surfaces. The prose chapters are curated, but every headline statement is expected to land on a theorem object, publication matrix, replay artifact, or explicit non-claim surface.

d.1 Claim classes and governing surfaces

Table d.1: Claim classes and their governing artifact surfaces.

Claim class	Primary surface	What governs it
Proved	Typed universal objects, assumption register, proof appendix	Theorems must point to assumptions, typed runtime objects, and the theorem-to-artifact bridge.
Validated	Publication matrices, replay tables, domain closure rows	Claims may advance only when the corresponding replay, calibration, runtime, and governance artifacts exist.
Governance	CertOS lifecycle surfaces, runtime/governance matrices	Certificate lifecycle, fallback semantics, and audit continuity remain first-class evidence, not narrative-only claims.
Roadmap	Explicit non-claims, parity blockers, promotion obligations	Future work is useful only when it stays outside headline proof or validation language until artifacts close the gap.

d.2 Headline claim families

Table d.2: Headline claim families and their canonical evidence surfaces.

Claim family	Chapter anchor	Canonical evidence surfaces
Degraded-observation hazard	Chapters 1–3	Typed adapter semantics, dual-trajectory benchmark contract, and OASG-facing replay logic.
Theorem ladder	Chapter 5	Assumption register, proof appendix, theorem dependency table, integrated theorem gate figure.
Witness-domain validation	Chapters 7–8	Main results, ablations, grouped coverage tables, learned-reliability surfaces, runtime traces.
Cross-domain universality	Chapter 9	Parity matrix, domain closure matrix, cross-domain validation figure, runtime/-governance matrices.
Temporal and governance semantics	Chapters 10 and 12	Blackout half-life figure, graceful degradation figure, governance lifecycle matrix, runtime budget matrix.
Exact non-claims and parity blockers	Chapter 13	Submission scorecard, deployment validation scope, 93-plus gap matrix, parity matrix.

d.3 Canonical manuscript-facing evidence

The following generated surfaces define the defended manuscript state at any given time:

- `reports/publication/orius_equal_domain_parity_matrix.csv`
- `reports/publication/orius_submission_scorecard.csv`
- `reports/publication/orius_governance_lifecycle_matrix.csv`
- `reports/publication/orius_runtime_budget_matrix.csv`
- `reports/publication/orius_deployment_validation_scope.csv`
- `reports/publication/orius_93plus_gap_matrix.csv`

d.4 Interpretation

The canonical readiness summary lives in Chapter 9 because parity promotion is part of the main scientific narrative, not an appendix-only afterthought. This appendix records the governing surfaces that support those statements and keeps exact non-claims visible so the dissertation cannot silently promote a bounded row into a stronger evidence class.

Appendix e

Safety-Case Bundle Index

This appendix maps the dissertation to a safety-case style evidence bundle without making any compliance claim. The goal is simple: a reviewer should be able to see which hazard, which argument, which evidence packet, and which non-claim boundary belong together. The appendix is therefore an evidence-structure aid, not a sector-specific certification claim [9–12].

Table e.1: Safety-case bundle index for the ORIUS dissertation.

Bundle	**Dissertation role**	**Primary evidence surfaces**
Hazard definition	Defines degraded observation and OASG as the governing safety problem.	Chapters 1–3, universal claim matrix, assumption register.
Theorem bundle	States what is proved under explicit assumptions.	Chapter 5, Appendix b, theorem dependency table, integrated theorem gate figure.
Benchmark bundle	Defines replay, dual trajectories, metrics, and fault families.	Chapter 6, Appendix c, domain closure matrix, parity matrix.
Witness bundle	Shows the deepest theorem-to-artifact row.	Chapters 7–8, battery tables and figures, release manifest.
Cross-domain bundle	Shows structural universality and bounded transfer.	Chapter 9, equal-domain parity matrix, submission scorecard, runtime/governance matrices.

Bundle	Dissertation role	Primary evidence surfaces
Governance bundle	Shows certificate lifecycle, runtime budgets, fallback, and audit continuity.	Chapter 12, governance lifecycle matrix, runtime budget matrix, review-facing governance derivatives.
Boundary bundle	States what remains gated, roadmap, or non-promotional.	Chapter 13, deployment validation scope, gap matrix, exact non-claim prose.

e.1 How to read the bundle index

The bundle index is intentionally orthogonal to the chapter order. A chapter can contribute to more than one bundle, and a bundle can span multiple generated artifacts. That structure mirrors real safety-argument practice more closely than a chapter-by-chapter checklist: one claim family may rely simultaneously on a theorem statement, a replay trace, a runtime lifecycle table, and an explicit non-claim boundary.

e.2 Interpretive boundary

The dissertation uses this bundle index only as an evidence-structure aid. It aligns the manuscript with disciplined safety-argument practice and lifecycle-oriented risk governance without implying certification or sector-specific regulatory compliance. Its role is to keep the argument inspectable, not to relabel the dissertation as a compliance submission.

Extended Reading and Index

Suggested Extended Reading

Universal Safety Foundations

Angelopoulos and Bates, *Conformal Prediction: A Gentle Introduction* (2023); Vovk, Gammerman, and Shafer, *Algorithmic Learning in a Random World* (2005); Shafer and Vovk, "A Tutorial on Conformal Prediction" (JMLR 2008).

Conformal Prediction and Calibration

Romano, Patterson, and Candès, "Conformalized Quantile Regression" (NeurIPS 2019); Gibbs and Candès, "Adaptive Conformal Inference Under Distribution Shift" (NeurIPS 2021); Barber et al., "Conformal Prediction Beyond Exchangeability" (AoS 2023); Tibshirani et al., "Conformal Prediction Under Covariate Shift" (2019); Zaffran et al., "Adaptive Conformal Predictions for Time Series" (2022).

Runtime Assurance and Safe Control

Ames et al., "Control Barrier Functions: Theory and Applications" (2019); Bloem et al., "Shield Synthesis" (TACAS 2015); Wabersich and Zeilinger, "A Predictive Safety Filter for Learning-Based Control" (2021); Alshiekh et al., "Safe Reinforcement Learning via Shielding" (2018); Desai et al., "SOTER" (IROS 2019).

Degraded Observation and CPS Trust

Chandola, Banerjee, and Kumar, "Anomaly Detection: A Survey" (2009); Liu, Ting, and Zhou, "Isolation Forest" (2008); Teixeira et al., "A Secure Control Framework for Resource-Limited Adversaries" (2015); Cardenas et al., "Attacks Against Process Control Systems" (2011); Giraldo et al., "A Survey of Physics-Based Attack Detection in Cyber-Physical Systems" (2018).

Domain Instantiations

Energy and storage: Tarascon and Armand (2001), Dunn et al. (2011), Rosewater et al. (2019), Hossain et al. (2020). Automated driving and robotics: Koopman and

Wagner (2016), Shalev-Shwartz et al. (2017), Althoff and Dolan (2014), Herbert et al. (2017). Industrial systems: Qin and Badgwell (2003), Yin et al. (2014), Isermann (2005), Zhang and Jiang (2008). Healthcare CPS: Lee and Sokolsky (2010), Saeed et al. (2011), Clifford et al. (2009), FDA SaMD guidance (2019). Navigation and planning: Kaelbling, Littman, and Cassandra (1998), Julier and Uhlmann (2004), Rockafellar and Uryasev (2000). Aerospace assurance: RTCA DO-178C (2011), Rushby (1993), Habli and Kelly (2010), Srinivasan and Sanner (2020).

Reproducibility and Scientific Software

Gneiting and Raftery (2007); Makridakis et al. (2020); Lei et al. (2018); Barber, Candès, Ramdas, and Tibshirani (2023). Use the locked artifact index and publication manifests for all thesis-grade claims.

Key Symbol Index

The dissertation keeps its canonical notation in the core theory and assumption surfaces: see Chapters 3–5 and Appendix a.

Artifact Index

The artifact, figure, and table index is maintained in the repository at `reports/publication/` alongside the locked evaluation outputs.

Bibliography

[1] V. Vovk, A. Gammerman, and G. Shafer, "Algorithmic learning in a random world," 2005, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[2] Y. Romano, E. Patterson, and R. J. Candès, "Conformalized quantile regression," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[3] I. Gibbs and E. J. Candès, "Adaptive conformal inference under distribution shift," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[4] Seto, Daisuke and Krogh, Bruce H. and Sha, Lui and Chutinan, Alongkrit, "The simplex architecture for safe online control system upgrades," 1998, proceedings of ACC.

[5] Sha, Lui, "Using simplicity to control complexity," 2001, iEEE Software.

[6] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[7] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," 2014, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[8] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[9] UL Standards and Engagement, "Ul 4600: Standard for safety for the evaluation of autonomous products," 2023, underwriters Laboratories standard.

[10] National Institute of Standards and Technology, "Artificial intelligence risk management framework (ai rmf 1.0)," 2023, nIST AI governance framework.

[11] International Organization for Standardization, "Iso 26262 road vehicles – functional safety," 2018, international functional-safety standard.

[12] International Electrotechnical Commission, "Iec 61508 functional safety of electrical, electronic, and programmable electronic safety-related systems," 2010, cross-sector functional-safety standard.

[13] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-free predictive inference for regression," 2018, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[14] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," 2023, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[15] R. F. Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," 2023, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[16] M. Zaffran, O. Féron, Y. Goude, J. Jolivet, and A. Dieuleveut, "Adaptive conformal predictions for time series," 2022, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[17] K. Stankeviciute, A. M'Charrak, R. Cornish, and T. Rainforth, "Conformal time-series forecasting," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[18] Xu, Chen and Xie, Yao, "Conformal prediction interval for dynamic time-series," 2021, proceedings of ICML.

[19] Sesia, Matteo and Candes, Emmanuel J. and Romano, Yaniv, "A comparison of some conformal quantile regression methods," 2021, stat.

[20] Bates, Stephen and Candes, Emmanuel J. and Lei, Lihua and Romano, Yaniv, "Distribution-free, risk-controlling prediction sets," 2021, journal of the ACM.

[21] Desai, Ankush and Ghosal, Akash and Saha, Indranil and Yang, Yuhong and Dutle, Aaron and Ivanov, Radoslav and Lee, Insup and Seshia, Sanjit A. and Tiwari, Ashish, "Soter: A runtime assurance framework for programming safe robotics systems," 2019, proceedings of IROS.

[22] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield synthesis: Runtime enforcement for reactive systems," 2015, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[23] Bartocci, Ezio and Falcone, Ylies and Francalanza, Adrian and Reger, Giles, "Introduction to runtime verification," 2018, lecture Notes in Computer Science.

[24] Havelund, Klaus and Rosu, Grigore, "An overview of the runtime verification tool java pathexplorer," 2004, formal Methods in System Design.

[25] M. Leucker and C. Schallhart, "A brief account of runtime verification," 2009, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[26] Schierman, John D. and Wang, Xun J. and Griffith, Dustin and DeCastro, Jonathan A. and Dutle, Aaron and Pike, Lee and Johnson, Taylor T., "Run-time assurance for complex systems," 2015, aIAA Infotech@Aerospace.

[27] Bak, Stanley and Manamcheri, Krishna and Mitra, Sayan and Caccamo, Marco, "Sandboxing controllers for cyber-physical systems," 2011, proceedings of ICCPS.

[28] Nguyen, Quan and Sreenath, Koushil, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," 2016, proceedings of ACC.

[29] Wang, Li and Ames, Aaron D. and Egerstedt, Magnus, "Safety barrier certificates for collisions-free multirobot systems," 2017, iEEE Transactions on Robotics.

[30] Fisac, Jaime F. and Akametalu, Anayo and Zeilinger, Melanie N. and Kaynama, Sahar and Gillula, Jeremy and Tomlin, Claire J., "A general safety framework for learning-based control in uncertain robotic systems," 2019, iEEE Transactions on Automatic Control.

[31] D. Q. Mayne, M. M. Seron, and S. V. Raković, "Robust model predictive control of constrained linear systems with bounded disturbances," 2005, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[32] W. Langson, I. Chryssochoos, S. V. Rakovič, and D. Q. Mayne, "Robust model predictive control using tubes," 2004, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[33] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, "Model predictive control: Theory, computation, and design," 2017, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[34] Ben-Tal, Aharon and El Ghaoui, Laurent and Nemirovski, Arkadi, "Robust optimization," 2009, princeton University Press.

[35] Camacho, Eduardo F. and Bordons, Carlos, "Model predictive control," 2013, springer.

[36] Rajkumar, Ragunathan and Lee, Insup and Sha, Lui and Stankovic, John, "Cyber-physical systems: The next computing revolution," 2010, proceedings of DAC.

[37] Derler, Patricia and Lee, Edward A. and Sangiovanni-Vincentelli, Alberto, "Modeling cyber-physical systems," 2012, proceedings of the IEEE.

[38] Baheti, Radhakisan and Gill, Helen, "Cyber-physical systems," 2011, the Impact of Control Technology.

[39] Pasqualetti, Fabio and Dorfler, Florian and Bullo, Francesco, "Attack detection and identification in cyber-physical systems," 2013, iEEE Transactions on Automatic Control.

[40] A. Teixeira, I. Shames, H. Sandberg, and K. H. Johansson, "A secure control framework for resource-limited adversaries," 2015, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[41] Basseville, Michele and Nikiforov, Igor V., "Detection of abrupt changes: Theory and application," 1993, prentice Hall.

[42] Page, E. S., "Continuous inspection schemes," 1954, biometrika.

[43] ——, "A test for a change in a parameter occurring at an unknown point," 1955, biometrika.

[44] Hinkley, David V., "Inference about the change-point in a sequence of random variables," 1971, biometrika.

[45] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," 2008, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[46] Breunig, Markus M. and Kriegel, Hans-Peter and Ng, Raymond T. and Sander, Jorg, "Lof: Identifying density-based local outliers," 2000, proceedings of SIGMOD.

[47] Y. Romano, E. Patterson, and R. J. Candès, "Conformalized quantile regression," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[48] I. Gibbs and E. J. Candès, "Adaptive conformal inference under distribution shift," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[49] V. Vovk, A. Gammerman, and G. Shafer, "Algorithmic learning in a random world," 2005, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[50] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," 2016, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[51] G. Ke *et al.*, "Lightgbm: A highly efficient gradient boosting decision tree," 2017, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[52] B. N. Oreshkin, D. Carpóv, N. Chapados, and Y. Bengio, "N-beats: Neural basis expansion analysis for interpretable time series forecasting," 2020, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[53] B. Lim, S. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[54] Y. Nie *et al.*, "A time series is worth 64 words: Long-term forecasting with transformers," 2023, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[55] A. Wachi, Y. Sui, Y. Yue, and M. Ono, "Safe reinforcement learning in constrained markov decision processes," 2024, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[56] D. Amodei *et al.*, "Concrete problems in ai safety," 2016, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[57] A. Bemporad and M. Morari, "Robust model predictive control: A survey," 2007, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[58] A. A. Cardenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," 2011, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[59] Y. Liang, C.-Z. Zhao, H. Yuan, Z. Chen, W. Zhang, J.-Q. Huang, D. Yu, Y. Liu, M.-M. Titirici, Y.-L. Chueh, H. Yu, and Q. Zhang, "A review of rechargeable batteries

for portable electronic devices," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[60] A. Shapiro, D. Dentcheva, and A. Ruszczyński, "Lectures on stochastic programming: Modeling and theory," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[61] H. Sun, J. Chen, Q. Ye, and J. Shao, "Adversarial attacks on neural networks for logical reasoning," 2007, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[62] C. Xu and Y. Xie, "Conformal prediction interval for dynamic time-series," 2021, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[63] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," 2015, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[64] R. J. Tibshirani, B. Efron, R. F. Barber, E. J. Candès, and A. Ramdas, "Conformal prediction under covariate shift," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[65] H. Papadopoulos, K. Proedrou, V. Vovk, and A. Gammerman, "Inductive confidence machines for regression," 2002, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[66] S. Bates, E. J. Candès, L. Lei, Y. Romano, and M. Sesia, "Testing for outliers with conformal p-values," 2023, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[67] S. Prajna and A. Jadbabaie, "Safety verification of hybrid systems using barrier certificates," 2004, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[68] A. J. Taylor and A. D. Ames, "Adaptive safety with control barrier functions," 2020, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[69] P. Wieland and F. Allgöwer, "Constructive safety using control barrier functions," 2007, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[70] S. V. Raković, E. C. Kerrigan, K. I. Kouramas, and D. Q. Mayne, "Invariant approximations of the minimal robust positively invariant set," 2005, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[71] M. Lorenzen, F. Dabbene, R. Tempo, and F. Allgöwer, "Constraint-tightening and stability in stochastic model predictive control," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[72] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," 2017, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[73] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," 2022, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[74] T. M. Moldovan and P. Abbeel, "Safe exploration in markov decision processes," 2012, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[75] A. Ray, J. Achiam, and D. Amodei, "Benchmarking safe exploration in deep reinforcement learning," 1910, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[76] E. Altman, "Constrained markov decision processes," 1999, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[77] Y. Falcone, K. Havelund, and G. Reger, "A tutorial on runtime verification," 2013, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[78] E. Bartocci, Y. Falcone, A. Francalanza, and G. Reger, "Introduction to runtime verification," 2018, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[79] A. Pnueli and A. Zaks, "Psl model checking and run-time verification via testers," 2006, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[80] O. Maler and D. Nickovic, "Monitoring temporal properties of continuous signals," 2004, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[81] D. I. Urbina, J. A. Giraldo, A. A. Cardenas, N. O. Tippenhauer, J. Valente, M. Faisal, J. Ruths, R. Candell, and H. Sandberg, "Limiting the impact of stealthy attacks on

industrial control systems," 2016, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[82] Y. Mo and B. Sinopoli, "Secure estimation in the presence of integrity attacks," 2015, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[83] J. A. Giraldo, D. Urbina, A. A. Cardenas, J. Valente, M. Faisal, J. Ruths, N. O. Tippenhauer, H. Sandberg, and R. Candell, "A survey of physics-based attack detection in cyber-physical systems," 2018, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[84] X. Hu, S. Li, and H. Peng, "A comparative study of equivalent circuit models for li-ion batteries," 2012, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[85] J. M. Tarascon and M. Armand, "Issues and challenges facing rechargeable lithium batteries," 2001, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[86] B. Dunn, H. Kamath, and J.-M. Tarascon, "Electrical energy storage for the grid: A battery of choices," 2011, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[87] S. Bose, S. Fattahi, and A. Swami, "Stochastic battery operation in a competitive electricity market," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[88] D. M. Rosewater, D. A. Copp, T. A. Nguyen, R. H. Byrne, and S. Santoso, "Battery energy storage models for optimal control," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[89] T. Hong and S. Fan, "Probabilistic electric load forecasting: A tutorial review," 2016, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[90] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," 2007, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[91] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The m4 competition: 100,000 time series and 61 forecasting methods," 2020, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[92] J. W. Taylor, "Short-term electricity demand forecasting using double seasonal exponential smoothing," 2003, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[93] P. Koopman and M. Wagner, "Autonomous vehicle safety: An interdisciplinary challenge," 2016, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[94] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a formal model of safe and scalable self-driving cars," 2017, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[95] M. Althoff and J. M. Dolan, "Online verification of automated road vehicles using reachability analysis," 2014, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[96] T. Nolte, M. Sjodin, K. Akesson, K. Lundqvist, M. Behnam, M. Lindberg, and C. Normann, "Towards bringing research on functional safety in automotive systems to practice," 2017, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[97] I. Lee and O. Sokolsky, "Medical cyber physical systems," 2010, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[98] M. Saeed, M. Villarroel, A. T. Reisner, G. Clifford, L. W. Lehman, G. Moody, T. Heldt, T. H. Kyaw, B. Moody, and R. G. Mark, "Multiparameter intelligent monitoring in intensive care ii (mimic-ii): A public-access intensive care unit database," 2011, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[99] G. D. Clifford, W. J. Long, G. B. Moody, and P. Szolovits, "Robust parameter extraction for decision support using multimodal intensive care data," 2009, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[100] U.S. Food and Drug Administration, "Software as a medical device (samd): Clinical evaluation," 2019, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[101] RTCA, "Do-178c: Software considerations in airborne systems and equipment certification," 2011, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[102] J. Rushby, "Formal methods and the certification of critical systems," 1993, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[103] I. Habli and T. Kelly, "Model-based assurance of control system safety and security," 2010, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[104] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," 1998, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[105] S. J. Julier and J. K. Uhlmann, "Unscented filtering and nonlinear estimation," 2004, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[106] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," 2000, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[107] S. J. Qin and T. A. Badgwell, "A survey of industrial model predictive control technology," 2003, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[108] S. Yin, S. X. Ding, X. Xie, and H. Luo, "A review on basic data-driven approaches for industrial process monitoring," 2014, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[109] R. Isermann, "Model-based fault-detection and diagnosis—status and applications," 2005, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[110] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," 2009, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[111] A. B. Tsybakov, "Introduction to nonparametric estimation," 2009, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[112] B. Yu, "Assouad, fano, and le cam," 1997, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[113] M. Blanke, M. Kinnaert, J. Lunze, and M. Staroswiecki, "Diagnosis and fault-tolerant control," 2006, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[114] Y. Zhang and J. Jiang, "Bibliographical review on reconfigurable fault-tolerant control systems," 2008, imported from the legacy ORIUS bibliography surface in backmatter/references.tex.

[115] Chernozhukov, Victor and Wuthrich, Kaspar and Zhu, Yinchu, "Distributional conformal prediction," 2021, proceedings of the National Academy of Sciences.

[116] Feldman, Shai and Bates, Stephen and Candes, Emmanuel J., "Calibrated multiple-output quantile regression with conformal prediction," 2021, arXiv preprint.

[117] Fisch, Adam and Grimson, W. Eric and Vovk, Vladimir and Katz-Samuels, Julian, "Online conformal prediction with decaying step sizes," 2021, arXiv preprint.

[118] Cofer, Darren and Rangarajan, Anand and Janssen, Wil and Havelund, Klaus and Perez, Ivan, "Assurance cases for runtime assurance frameworks," 2012, nASA Technical Report.

[119] Herbert, Sylvia and Chen, Mo and Han, Siyuan and Bansal, Somil and Fisac, Jaime F. and Tomlin, Claire, "Fastrack: A modular framework for fast and guaranteed safe motion planning," 2017, proceedings of CDC.

[120] Sen, Koushik and Vardhan, Abhay and Agha, Gul and Rosu, Grigore, "Efficient decentralized monitoring of safety in distributed systems," 2004, proceedings of ICSE.

[121] d'Angelo, Ben and Sankaranarayanan, Sriram and Sanchez, Cesar and Robinson, Will and Finkbeiner, Bernd and Sipma, Henny and Mehrotra, Sharad and Manna, Zohar, "Lola: Runtime monitoring of synchronous systems," 2005, tIME.

[122] Xu, Xiangru and Tabuada, Paulo and Grizzle, Jessy W. and Ames, Aaron D., "Robustness of control barrier functions for safety critical control," 2015, iFAC-PapersOnLine.

[123] Cheng, Richard and Orosz, Gyorgy and Murray, Richard M. and Burdick, Joel W., "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," 2019, proceedings of AAAI.

[124] Cheng, Richard and Nurnberger, Andrew and Scheibler, Karol and Ivanov, Radoslav and Pappas, George J., "Cautious adaptation for reinforcement learning under safety constraints," 2021, proceedings of L4DC.

[125] Wabersich, Kai P. and Zeilinger, Melanie N. and Hewing, Lukas and Carron, Andrea, "Data-driven safety filters: A survey and perspective," 2023, annual Review of Control, Robotics, and Autonomous Systems.

[126] Koller, Torsten and Berkenkamp, Felix and Turchetta, Matteo and Krause, Andreas, "Learning-based model predictive control for safe exploration," 2018, proceedings of CDC.

[127] Berkenkamp, Felix and Turchetta, Matteo and Schoellig, Angela P. and Krause, Andreas, "Safe model-based reinforcement learning with stability guarantees," 2017, advances in Neural Information Processing Systems.

[128] Mo, Yilin and Sinopoli, Bruno, "False data injection attacks in control systems," 2012, proceedings of the First Workshop on Secure Control Systems.

[129] Urbina, David I. and Giraldo, Jairo and Cardenas, Alvaro A. and Tippenhauer, Nils Ole, "Survey and new directions for physics-based attack detection in control systems," 2016, proceedings of CPS-SPC.

[130] Khraisat, Ansam and Gondal, Iqbal and Vamplew, Peter and Kamruzzaman, Joarder, "Survey of intrusion detection systems: Techniques, datasets and challenges," 2019, cybersecurity.

[131] Salinas, David and Flunkert, Valentin and Gasthaus, Jan and Januschowski, Tim, "Deepar: Probabilistic forecasting with autoregressive recurrent networks," 2020, international Journal of Forecasting.

[132] Wu, Haixu and Xu, Jiehui and Wang, Jianmin and Long, Mingsheng, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," 2021, advances in Neural Information Processing Systems.

[133] Zhou, Haoyi and Zhang, Shanghang and Peng, Jieqi and Zhang, Shuai and Li, Jianxin and Xiong, Hui and Zhang, Wancai, "Informer: Beyond efficient transformer for long sequence time-series forecasting," 2021, proceedings of AAAI.

[134] Zhou, Tian and Ma, Ziqing and Wen, Qingsong and Wang, Xue and Sun, Liang and Jin, Rong and Zhou, Xiaokang and Qian, Wending and Li, Xue and Zhang, Weifeng and others, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," 2022, proceedings of ICML.

[135] Wu, Haixu and Hu, Tao and Liu, Yong and Zhou, Hang and Wang, Jianmin and Long, Mingsheng, "Timesnet: Temporal 2d-variation modeling for general time series analysis," 2023, proceedings of ICLR.

[136] Zeng, Ailing and Chen, Muxin and Zhang, Lei and Xu, Qiang, "Are transformers effective for time series forecasting?" 2023, proceedings of AAAI.

[137] Dosovitskiy, Alexey and Beyer, Lucas and Kolesnikov, Alexander and others, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021, proceedings of ICLR.

[138] Vaswani, Ashish and Shazeer, Noam and Parmar, Niki and Uszkoreit, Jakob and Jones, Llion and Gomez, Aidan and Kaiser, Lukasz and Polosukhin, Illia, "Attention is all you need," 2017, advances in Neural Information Processing Systems.

[139] Severson, Kristen A. and Attia, Peter M. and Jin, Norman and Perkins, Nicholas and Jiang, Ben and Yang, Zi and Chen, Michael and Aykol, Muratahan and Herring, Patrick K. and Fraggedakis, Dimitrios and others, "Data-driven prediction of battery cycle life before capacity degradation," 2019, nature Energy.

[140] Berecibar, Mikel and Gandiaga, Iker and Villarreal, Ibon and Omar, Noshin and Van Mierlo, Joeri and Van den Bossche, Peter, "Critical review of state of health estimation methods of li-ion batteries for real applications," 2016, renewable and Sustainable Energy Reviews.

[141] He, Hongwen and Xiong, Rui and Fan, Jianxin, "Evaluation of lithium-ion battery equivalent circuit models for state of charge estimation," 2011, applied Energy.

[142] Hu, Xiaosong and Li, Shengbo Eben and Peng, Huei, "A comparative study of equivalent circuit models for li-ion batteries," 2012, journal of Power Sources.

[143] Weitzel, Tim and Glock, Christoph H., "Energy storage dispatch in energy markets: A review," 2018, energy.

[144] Xu, Bing and Oudalov, Alexandre and Ulbig, Andreas and Andersson, Goran and Kirschen, Daniel S., "Modeling of lithium-ion battery degradation for cell life assessment," 2020, iEEE Transactions on Smart Grid.

[145] Wang, Yi and Zhang, Dong and Li, Xinyu, "Battery storage operation under uncertainty: A distributionally robust perspective," 2022, iEEE Transactions on Smart Grid.

[146] Hossain, Eklas and Faruque, Hasan and Sunny, Md Shahjalal and Mohammad, Naimul and Nawar, Nurul, "A comprehensive review on grid-connected battery energy storage systems," 2020, energies.

[147] Plett, Gregory L., "Battery management systems, volume ii: Equivalent-circuit methods," 2015, artech House.

[148] ——, "Battery management systems, volume iii: Battery state estimation," 2016, artech House.

[149] Srinivasan, Ranjani and Sanner, Scott and others, "Runtime assurance for safe learning-enabled aerospace systems," 2020, aIAA Scitech.

[150] Herbert, Sylvia and Chen, Mo and Han, Siyuan and Bansal, Somil and Fisac, Jaime and Tomlin, Claire, "Fastrack: A modular framework for fast and guaranteed safe motion planning," 2017, proceedings of CDC.

[151] Ames, Aaron D. and Molnar, Tamas and Orosz, Gabor and Sreenath, Koushil and Tabuada, Paulo, "Safety-critical control for autonomous systems," 2021, annual Review of Control, Robotics, and Autonomous Systems.

[152] Lederer, Johannes and Kohler, Jonas and Berkenkamp, Felix and Zeilinger, Melanie N., "A survey on safe reinforcement learning," 2019, arXiv preprint.

[153] Ulbrich, Simon and Reschka, Andreas and Rieken, Jonathan and Ernst, Sebastian and Bagschik, Gereon and Damm, Werner and Maurer, Markus, "Towards a functional system architecture for automated vehicles," 2015, proceedings of ITSC.

[154] Schwarting, Wilko and Alonso-Mora, Javier and Rus, Daniela, "Planning and decision-making for autonomous vehicles," 2018, annual Review of Control, Robotics, and Autonomous Systems.

[155] Badue, Claudine and Guidolini, Rafael and Carneiro, Raphael and Berriel, Rodrigo and Paixao, Thiago and Mutz, Filipe and de Paula Veronese, Luis and Oliveira-Santos, Thiago, "Self-driving cars: A survey," 2021, expert Systems with Applications.

[156] Paszke, Adam and Gross, Sam and Massa, Francisco and others, "Pytorch: An imperative style, high-performance deep learning library," 2019, advances in Neural Information Processing Systems.

[157] Sutton, Richard S. and Barto, Andrew G., "Reinforcement learning: An introduction," 2018, mIT Press.

[158] Kulkarni, Saurabh and Sahoo, Sarmila and Mohanty, Sasmita, "Industrial process monitoring and fault diagnosis: A survey," 2022, computers and Chemical Engineering.

[159] Yin, Shen and Ding, Steven and Xie, Xiaoxia and Luo, Haifeng, "A review on basic data-driven approaches for industrial process monitoring," 2014, iEEE Transactions on Industrial Electronics.

[160] Chiang, Leo H. and Russell, Edwin L. and Braatz, Richard D., "Fault detection and diagnosis in industrial systems," 2001, springer.

[161] Clifton, Lei and Clifton, David A. and Pimentel, Marco A. F. and Watkinson, Peter J. and Tarassenko, Lionel, "Gaussian processes for personalized e-health monitoring," 2012, iEEE Transactions on Biomedical Engineering.

[162] Goldstein, Benjamin A. and Navar, Ann Marie and Pencina, Michael J. and Ioannidis, John P. A., "Opportunities and challenges in developing risk prediction models with electronic health records data," 2017, journal of the American Medical Informatics Association.

[163] Johnson, Alistair E. W. and Pollard, Tom J. and Shen, Lu and Lehman, Li-wei H. and Feng, Mengling and Ghassemi, Mohammad and Moody, Benjamin and Szolovits, Peter and Celi, Leo Anthony and Mark, Roger G., "Mimic-iii, a freely accessible critical care database," 2016, scientific Data.

[164] Esteva, Andre and Robicquet, Alexandre and Ramsundar, Bharath and Kuleshov, Volodymyr and DePristo, Mark and Chou, Katie and Cui, Claire and Corrado, Greg and Thrun, Sebastian and Dean, Jeff, "A guide to deep learning in healthcare," 2019, nature Medicine.

[165] Lee, Edward A., "The past, present and future of cyber-physical systems: A focus on models," 2015, sensors.

[166] Wolf, Mark T. and others, "Safety and trustworthiness in physical ai systems," 2018, aAAI Spring Symposium.

[167] Gernstedt, Erik and Nilsson, Ola and Smith, Reid, "Physical ai systems: Architectures, risks, and safety layers," 2024, arXiv preprint.

[168] Astrom, Karl Johan and Murray, Richard M., "Feedback systems: An introduction for scientists and engineers," 2008, princeton University Press.

[169] Zhou, Kemin and Doyle, John C. and Glover, Keith, "Robust and optimal control," 1996, prentice Hall.

[170] Bertsekas, Dimitri P., "Dynamic programming and optimal control," 2005, athena Scientific.

[171] Vovk, Vladimir and Gammerman, Alex and Shafer, Glenn, "Conformal predictors and confidence machines," 2005, algorithmic Learning in a Random World.

[172] Shafer, Glenn and Vovk, Vladimir, "A tutorial on conformal prediction," 2008, journal of Machine Learning Research.

[173] Barber, Rina Foygel and Candes, Emmanuel J. and Ramdas, Aaditya and Tibshirani, Ryan J., "Conformal prediction beyond exchangeability," 2023, annals of Statistics.

[174] Alshiekh, Mohammad and Bloem, Roderick and Ehlers, Ruediger and Konighofer, Bettina and Niekum, Scott and Topcu, Ufuk, "Safe reinforcement learning via shielding," 2018, proceedings of AAAI.

[175] Konighofer, Bettina and Alshiekh, Mohammad and Bloem, Roderick, "Shielding techniques for safe reinforcement learning," 2020, formal Methods in System Design.

[176] Bastani, Osbert, "Safe reinforcement learning: A control-theoretic perspective," 2021, foundations and Trends in Machine Learning.

[177] Bertsimas, Dimitris and Brown, David B. and Caramanis, Constantine, "Theory and applications of robust optimization," 2011, sIAM Review.

[178] Birge, John R. and Louveaux, Francois, "Introduction to stochastic programming," 2011, springer.

[179] Borisov, Vadim and Leemann, Tobias and Seßler, Kathrin and Haug, Johannes and Pawelczyk, Martin and Kasneci, Gjergji, "Deep neural networks and tabular data: A survey," 2022, iEEE Transactions on Neural Networks and Learning Systems.

[180] Chen, Min and Lu, Shengnan and Wang, Zhen and Wang, Xin, "A review of battery energy storage system safety and reliability," 2020, journal of Energy Storage.

[181] Dibaji, Seyed Mohammad and Pirani, Mohsen and Flamholz, David and Annaswamy, Anuradha and Johansson, Karl Henrik and Chakrabortty, Aranya, "A systems and control perspective of cps security," 2019, annual Reviews in Control.

[182] Lamport, Leslie and Shostak, Robert and Pease, Marshall, "The byzantine generals problem," 1982, aCM Transactions on Programming Languages and Systems.

[183] Lundberg, Scott M. and Lee, Su-In, "A unified approach to interpreting model predictions," 2017, advances in Neural Information Processing Systems.

[184] Pineau, Joelle and Vincent-Lamarre, Philippe and Sinha, Koustuv and Lariviere, Vincent and Beygelzimer, Alina and d'Alche-Buc, Florence and Fox, Emily and Larochelle, Hugo, "Improving reproducibility in machine learning research," 2021, journal of Machine Learning Research.

[185] Rosewater, David and Ferraro, Paul and Byrne, Raymond and Santoso, Surya, "Risk-averse battery dispatch under forecast error," 2020, iEEE Transactions on Smart Grid.

[186] Leveson, Nancy, "Engineering a safer world: Systems thinking applied to safety," 2011, mIT Press.

[187] Koopman, Philip and Wagner, Michael, "Challenges in autonomous vehicle testing and validation," 2016, sAE International Journal of Transportation Safety.

[188] Dawid, A. Philip, "The well-calibrated bayesian," 1982, journal of the American Statistical Association.

[189] Platt, John, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," 1999, advances in Large Margin Classifiers.

[190] Guo, Chuan and Pleiss, Geoff and Sun, Yu and Weinberger, Kilian Q., "On calibration of modern neural networks," 2017, proceedings of ICML.

[191] Kuleshov, Volodymyr and Fenner, Nathan and Ermon, Stefano, "Accurate uncertainties for deep learning using calibrated regression," 2018, proceedings of ICML.

[192] Vaicenavicius, Joris and Widmann, David and Andersson, Carl and Lindsten, Fredrik and Roll, Jacob and Sch"on, Thomas B., "Evaluating model calibration in classification," 2019, proceedings of AISTATS.

[193] Nixon, Jeremy and Dusenberry, Michael W. and Zhang, Linchuan and Jerfel, Ghassen and Tran, Dustin, "Measuring calibration in deep learning," 2019, proceedings of CVPR Workshops.

[194] Lee, Edward A., "Cyber physical systems: Design challenges," 2008, proceedings of ISORC.

[195] Kopetz, Hermann, "Real-time systems: Design principles for distributed embedded applications," 2011, springer.

[196] Alur, Rajeev, "Principles of cyber-physical systems," 2015, mIT Press.

[197] Blanchini, Franco, "Set invariance in control," 1999, automatica.

[198] Aubin, Jean-Pierre, "Viability theory," 1991, birkh"auser.

[199] Mitchell, Ian M. and Bayen, Alexandre M. and Tomlin, Claire J., "A time-dependent hamilton-jacobi formulation of reachable sets for continuous dynamic games," 2005, iEEE Transactions on Automatic Control.

[200] Baier, Christel and Katoen, Joost-Pieter, "Principles of model checking," 2008, mIT Press.

[201] Clarke, Edmund M. and Grumberg, Orna and Peled, Doron, "Model checking," 1999, mIT Press.

[202] Geiger, Andreas and Lenz, Philip and Urtasun, Raquel, "Are we ready for autonomous driving? the kitti vision benchmark suite," 2012, proceedings of CVPR.

[203] Caesar, Holger and Bankiti, Varun and Lang, Alex H. and Vora, Sourabh and Liong, Venice Erin and Xu, Qiang and Krishnan, Anush and Pan, Yu and Baldan, Giancarlo and Beijbom, Oscar, "nuscenes: A multimodal dataset for autonomous driving," 2020, proceedings of CVPR.

[204] Ettinger, Scott and Cheng, Shuyang and Caine, Benjamin and Liu, Chenxi and Zhao, Hang and Pradhan, Saurabh and Chai, Yuning and Sapp, Benjamin and Qi, Charles R. and Zhou, Yin and others, "Large scale interactive motion forecasting for autonomous driving: The waymo open motion dataset," 2021, proceedings of ICCV.

[205] Wilson, Benjamin and Qi, Wenda and Agarwal, Tushar and Lambert, John and Singh, Jitendra and Khandelwal, Siddhesh and Pan, Bowen and Wang, Cong and D'Sa, Sharath and Lee, Seung and others, "Argoverse 2: Next generation datasets for self-driving perception and forecasting," 2021, arXiv preprint.

[206] Downs, James J. and Vogel, Ernest F., "A plant-wide industrial process control problem," 1993, computers and Chemical Engineering.

[207] Rieth, Cory A. and Amsel, Ben D. and Tran, Rene and Cook, Miles B., "Revisiting the tennessee eastman process simulation benchmark for industrial machine learning," 2017, proceedings of the AIChE Annual Meeting.

[208] Goldberger, Ary L. and Amaral, Luis A. N. and Glass, Leon and Hausdorff, Jeffrey M. and Ivanov, Plamen Ch. and Mark, Roger G. and Mietus, Joseph E. and Moody, George B. and Peng, C.-K. and Stanley, H. Eugene, "Physiobank, physiotoolkit, and physionet: Components of a new research resource for complex physiologic signals," 2000, circulation.

[209] Johnson, Alistair E. W. and Bulgarelli, Lucas and Shen, Lu and Gayles, Anna and Shammout, Ahmad and Horng, Steven and Pollard, Tom and Hao, Shirly and Moody, Benjamin and Gow, Brian and others, "Mimic-iv, a freely accessible electronic health record dataset," 2023, scientific Data.

[210] Pollard, Tom J. and Johnson, Alistair E. W. and Raffa, Jonathan D. and Celi, Leo Anthony and Mark, Roger G. and Badawi, Omar, "The eicu collaborative research database, a freely available multi-center database for critical care research," 2018, scientific Data.

[211] Saxena, Abhinav and Goebel, Kai, "Turbofan engine degradation simulation data set," 2008, nASA Ames Prognostics Data Repository.

[212] Ramasso, Emmanuel and Saxena, Abhinav, "Performance benchmarking and analysis of prognostic methods for c-mapss datasets," 2014, international Journal of Prognostics and Health Management.

[213] Sch"afer, Matthias and Strohmeier, Martin and Olive, Xavier and Smith, Matthew and Lenders, Vincent and Martinovic, Ivan, "Opensky report 2020: Analysing in-flight emergency events using big data," 2020, openSky Network Report.

[214] Strohmeier, Martin and Sch"afer, Matthias and Lenders, Vincent and Martinovic, Ivan, "On perception and reality in wireless air traffic communication security," 2014, proceedings of IEEE CNS.

[215] Peng, Roger D., "Reproducible research in computational science," 2011, science.

[216] Sandve, Geir Kjetil and Nekrutenko, Anton and Taylor, James and Hovig, Eivind, "Ten simple rules for reproducible computational research," 2013, pLoS Computational Biology.

[217] Wilson, Greg and Aruliah, D. A. and Brown, C. Titus and Chue Hong, Neil P. and Davis, Matt and Guy, Richard T. and Haddock, Steven H. D. and Huff, Kathryn D. and Mitchell, Ian M. and Plumbley, Mark D. and others, "Best practices for scientific computing," 2014, pLoS Biology.

[218] Amershi, Saleema and Begel, Andrew and Bird, Christian and DeLine, Robert and Gall, Harald and Kamar, Ece and Nagappan, Nachiappan and Nushi, Besmira and Zimmermann, Thomas, "Software engineering for machine learning: A case study," 2019, proceedings of ICSE-SEIP.

[219] Doshi-Velez, Finale and Kim, Been, "Towards a rigorous science of interpretable machine learning," 2017, arXiv preprint.

[220] Rudin, Cynthia, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," 2019, nature Machine Intelligence.

[221] Lipton, Zachary C., "The mythos of model interpretability," 2018, queue.

[222] Russell, Stuart, "Human compatible: Artificial intelligence and the problem of control," 2019, viking.

[223] Koopman, Philip and Wagner, Michael, "Autonomous vehicle safety: An interdisciplinary challenge," 2017, iEEE Intelligent Transportation Systems Magazine.

[224] Favarò, Francesca M. and Eurich, Samantha O. and Nader, Nadine and Tripp, Matthew, "Examining accident reports involving autonomous vehicles in california," 2017, pLoS ONE.

[225] Kalra, Nidhi and Paddock, Susan M., "Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability?" 2016, transportation Research Part A.

[226] Cummings, Mary L., "Managing the complexity of autonomous vehicle safety," 2017, journal of Transportation Safety and Security.

[227] Dolan, John M. and others, "The field benchmark problem and evaluation standard for mobile robots," 1994, aAAI Mobile Robot Workshop.

[228] Thrun, Sebastian and Montemerlo, Michael and Dahlkamp, Hendrik and Stavens, David and Aron, Alex and Diebel, James and Fong, Philip and Gale, John and Halpenny, Morgan and Hoffmann, Gabe and others, "Stanley: The robot that won the darpa grand challenge," 2006, journal of Field Robotics.

[229] Montemerlo, Michael and Becker, Jan and Bhat, Surya and Dahlkamp, Hendrik and Dolgov, Dmitri and Ettinger, Scott and Haehnel, Dirk and Hilden, Tim and Hoffmann, Gabe and Huhnke, Burkhard and others, "Junior: The stanford entry in the urban challenge," 2008, journal of Field Robotics.

[230] Broggi, Alberto and Medici, Paolo and Cardarelli, Emanuele and Cerri, Pietro, "The vislab intercontinental autonomous challenge: An extensive road test of autonomous driving technologies," 2013, iEEE Intelligent Transportation Systems Magazine.

[231] LaValle, Steven M., "Planning algorithms," 2006, cambridge University Press.

[232] Kochenderfer, Mykel J., "Decision making under uncertainty: Theory and application," 2015, mIT Press.

[233] Rasmussen, Carl Edward and Williams, Christopher K. I., "Gaussian processes for machine learning," 2006, mIT Press.

[234] Murphy, Kevin P., "Machine learning: A probabilistic perspective," 2012, mIT Press.

[235] Jordan, Michael I. and Mitchell, Tom M., "Machine learning: Trends, perspectives, and prospects," 2015, science.